

STAT 444 Statistical Learning - Advanced Regression

University of Waterloo - Spring 2026

12th August 2026

Instructor: Michael John Davis

L^AT_EX: Xing Liu

Contents

1	Predictive Modelling	3
1.1	Setup	3
1.1.1	Square Loss and the Regression Function	5
1.1.2	Estimating the Regression Function	6
1.2	Model Assessment and Selection	8
1.2.1	Generalization Error	8
1.2.2	Bias-Variance Tradeoff	9
1.2.3	Optimism	9
1.2.4	Degrees of Freedom	11
1.2.5	Covariance Penalties	13
1.3	Cross-Validation	13
1.3.1	K-Fold Cross-Validation	14
1.3.2	Leave-One-Out Cross-Validation	15
1.3.3	Generalized Cross-Validation	18
2	Regularization and Penalized Regression	19
2.1	Linear Regression via Least Squares	19
2.1.1	Derivation and Properties	19
2.1.2	Distributional Properties	21

2.1.3	Spectral Analysis	22
2.2	Ridge Regression	23
2.2.1	Spectral Motivation	23
2.2.2	Penalized Least Squares	25
2.2.3	Penalty Parameter Selection by Generalized CV	27
3	Nonlinear Regression	29
3.1	Function Estimation	29
3.1.1	Estimating Functions is Impossible	29
3.1.2	Reproducing Kernel Hilbert Spaces (RKHS)	30
3.1.3	Minimal Norm Interpolation	34
3.1.4	Infinite-Dimensional Ridge Regression	35
3.1.5	Smoothing Splines: Theoretical Construction	36
3.2	Linear Basis Function Models	37
3.2.1	Polynomial Regression	38
3.2.2	Piecewise Polynomial Regression	38
3.3	Splines	41
3.3.1	B-Splines	42
3.3.2	Smoothing Splines: Practical Construction	43
3.4	Regression Splines	46
3.4.1	Fitting a Spline by Ridge Regression	47
3.4.2	Choosing λ	50
3.4.3	Penalty Nullspace	51
4	High-Dimensional Regression	53
4.1	Least Absolute Shrinkage and Selection Operator (LASSO)	53
4.1.1	L_1 -Penalties and Variable Selection	54
4.1.2	Coordinate Descent and Sparsity	54
4.2	Regression Trees	58
4.3	More Topics	60

1 Predictive Modelling

1.1 Setup

Input: $x \in \mathcal{X} \subseteq \mathbb{R}^p$ for $p \in \mathbb{N}$.

Output: $y \in \mathcal{Y} \subseteq \mathbb{R}$.

Training set: $\mathcal{T} = \{(x_1, y_1), \dots, (x_n, y_n)\} \in (\mathcal{X} \times \mathcal{Y})^n$ for $n \in \mathbb{N}$.

Goal: Given $\mathcal{T}$, find a mapping from $\mathcal{X}$ to $\mathcal{Y}$ that can predict the output y for a new input x .

Definition 1.1 (Learning Algorithm).

A **learning algorithm** is a function $\mathcal{A} : (\mathcal{X} \times \mathcal{Y})^n \rightarrow \{f : \mathcal{X} \rightarrow \mathcal{Y}\}$.

Note. The term “predictive modelling” usually describes a learning algorithm or a prediction rule, i.e. $f : \mathcal{X} \rightarrow \mathcal{Y}$, an output of a learning algorithm $\mathcal{A}$.

Example. For linear regression, the fitted prediction rule is

$$\hat{f}(x) = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \dots + \hat{\beta}_p x_p = \mathcal{A}((x_1, y_1), \dots, (x_n, y_n)).$$

Note that $\hat{\beta} = (X^\top X)^{-1} X^\top Y$.

Learning algorithms are evaluated by evaluating the quality of the predictions they produce. A learning algorithm is good if its predictions are generally close to the corresponding outputs.

Definition 1.2 (Loss Function).

A **loss function** is a function $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}^+ \cup \{0\}$ such that $\ell(y_1, y_2) = 0 \iff y_1 = y_2$.

Note. Lower values $\iff$ better predictions.

Example. The most common loss function is the **square loss**:

$$\ell(y_1, y_2) = (y_1 - y_2)^2 \geq 0.$$

We say $\mathcal{A}$ is good if it has small expected loss on a new input/output pair generated from the same distribution as the training set.

Suppose $(X, Y) \sim P_{X,Y}$ (some distribution) and suppose that $\mathcal{T} = ((X_1, Y_1), \dots, (X_n, Y_n)) \stackrel{\text{iid}}{\sim} P_{X,Y}$. Take the prediction rule $\mathcal{A}(\mathcal{T}) = \hat{f}$. Define a new point $(X_{n+1}, Y_{n+1}) \sim P_{X,Y}$ (test point). Then, the predicted output for the new input X_{n+1} is $\hat{Y}_{n+1} = \hat{f}(X_{n+1})$.

Definition 1.3 (Expected Loss).

The **expected loss** on a new point is

$$E = \int \ell(y_{n+1}, \hat{y}_{n+1}) dP_{X,Y}$$

which depends on $\mathcal{A}$, ℓ and is conditional on $\mathcal{T}$.

Note that we could use any loss function, but the choice of square loss is convenient. Here are some motivations and desirable properties of the square loss:

- (1) It leads to using the conditional mean of the output given the input as a learning algorithm.
- (2) It is the negative log-likelihood in a normal model where we predict using the mean, so it connects with classical statistics.
- (3) It approximates any loss function that is smooth and stationary.

For (3), suppose ℓ is continuous, 3-times differentiable and stationary. The stationary condition means $\ell(y_1, y_2) \equiv \ell(|y_1 - y_2|)$, i.e. the loss only depends on the difference between the predicted and true values. Observe that $\ell(y, y) = \ell(y - y) = 0$ and $\ell'(y, y) = \ell'(y - y) = 0$ (because minimum at 0). Let $h = y_1 - y_2 \implies \ell(y_1, y_2) = \ell(h)$. By Taylor's theorem,

$$\begin{aligned} \ell(h) &\approx \ell(0) + \ell'(0)h + \frac{1}{2}\ell''(0)h^2 \\ &= \frac{1}{2}\ell''(0)h^2. \end{aligned}$$

Also, $\ell''(0) > 0$ because 0 is a minimum. Thus, $\ell(h)$ is approximately a constant multiple of the square loss.

1.1.1 Square Loss and the Regression Function

The choice of a loss function leads to a specific prediction rule. We choose the prediction rule f that minimizes the expected loss:

$$\mathbb{E}[\ell(Y, f(X))] = \int \ell(y, f(x)) dP_{X,Y}$$

where $(X, Y) \sim P_{X,Y}$ and f is a prediction rule. The problem is to choose

$$f^* = \arg \min_f \mathbb{E}[\ell(Y, f(X))].$$

Suppose ℓ is the square loss: $\ell(Y, f(X)) = (Y - f(X))^2$. This leads to the following criterion for choosing f , i.e. the **expected prediction error** (EPE):

$$\begin{aligned} \text{EPE}(f) &= \mathbb{E}[(Y - f(X))^2] \\ &= \int [(y - f(x))^2] dP_{X,Y} \\ &= \mathbb{E}_X \mathbb{E}_{Y|X} [(Y - f(X))^2 | X] \quad (\text{law of total expectation}) \\ \implies f(x) &= \arg \min_c \mathbb{E}_{Y|X} [(Y - c)^2 | X = x]. \end{aligned}$$

The solution is the **regression function**:

$$f(x) = \mathbb{E}[Y | X = x].$$

Proof. We decompose the square loss:

$$\begin{aligned} (Y - f(x))^2 &= (Y - \mathbb{E}[Y | X = x] + \mathbb{E}[Y | X = x] - f(x))^2 \\ &= (Y - \mathbb{E}[Y | X = x])^2 + (\mathbb{E}[Y | X = x] - f(x))^2 \\ &\quad + 2(Y - \mathbb{E}[Y | X = x])(\mathbb{E}[Y | X = x] - f(x)). \end{aligned}$$

Recall that $\mathbb{E}[g(X, Y)] = \mathbb{E}_X [\mathbb{E}_{Y|X}[g(X, Y) | X]]$. Thus,

$$\begin{aligned}\mathbb{E}[(Y - f(X))^2] &= \mathbb{E}[\mathbb{E}[(Y - f(X))^2 | X]] \\ &= \mathbb{E}[\mathbb{E}[(Y - \mathbb{E}[Y | X])^2 + (\mathbb{E}[Y | X] - f(X))^2 + 2(Y - \mathbb{E}[Y | X])(\mathbb{E}[Y | X] - f(X)) | X]] \\ &= \mathbb{E}[\mathbb{E}[(Y - \mathbb{E}[Y | X])^2 | X] + (\mathbb{E}[Y | X] - f(X))^2].\end{aligned}$$

For a fixed value x , this becomes

$$\begin{aligned}\mathbb{E}[(Y - f(x))^2 | X = x] &= \mathbb{E}[(Y - \mathbb{E}[Y | X = x])^2 | X = x] + (\mathbb{E}[Y | X = x] - f(x))^2 \\ &\quad + 2\mathbb{E}[(Y - \mathbb{E}[Y | X = x])(\mathbb{E}[Y | X = x] - f(x)) | X = x].\end{aligned}$$

Note that $\forall x$,

$$\mathbb{E}[(Y - \mathbb{E}[Y | X = x])^2 | X = x] \geq 0$$

and

$$\mathbb{E}[(\mathbb{E}[Y | X = x] - f(x))^2 | X = x] \geq 0.$$

Also,

$$\begin{aligned}\mathbb{E}\left[(Y - \mathbb{E}[Y | X = x]) \underbrace{(\mathbb{E}[Y | X = x] - f(x))}_{\text{constant}} | X = x\right] &= (\mathbb{E}[Y | X = x] - f(x))\mathbb{E}[Y - \mathbb{E}[Y | X = x] | X = x] \\ &= (\mathbb{E}[Y | X = x] - f(x)) \cdot (\mathbb{E}[Y | X = x] - \mathbb{E}[Y | X = x]) \\ &= 0.\end{aligned}$$

Thus,

$$\mathbb{E}[(Y - f(x))^2 | X = x] = \mathbb{E}[(Y - \mathbb{E}[Y | X = x])^2 | X = x] + (\mathbb{E}[Y | X = x] - f(x))^2.$$

The first term does not depend on f . To minimize the expected loss, we need to minimize the second term, which is achieved by $f(x) = \mathbb{E}[Y | X = x]$. $\square$

1.1.2 Estimating the Regression Function

Given the training set $\mathcal{T}$, we want to estimate the regression function $f(x) = \mathbb{E}[Y | X = x]$. This involves:

- Choosing a model for the regression function.
- Fitting the model to the training data.

One option to estimate $f(x)$ is to use the law of large numbers or the central limit theorem:

$$\frac{1}{\#(X_i = x)} \sum_{i: X_i = x} Y_i \xrightarrow{\text{as } \#(X_i = x) \rightarrow \infty} \mathbb{E}[Y | X = x] \quad \forall x.$$

This has three problems:

- (1) The error in using the sample mean to approximate the population mean decreases with the number of data. So we will need lots of repeated inputs.
- (2) Cannot predict unseen inputs. Suppose x is a new input that does not appear in the training set. Then, $\#(X_i = x) = 0$. This means the method cannot extrapolate or interpolate.
- (3) No guarantee of smoothness/stability, i.e. predictions having similar inputs will be similar.

One way is to expand the range of data used in the prediction to any “nearby” inputs. This leads to **k-nearest neighbours** (KNN) prediction algorithm. For $k \in \mathbb{N}$, define

$$N_k(x) = \{k \text{ closest points to } x \text{ in the training set}\}.$$

Then, the KNN prediction is

$$\hat{f}_k(x) = \frac{1}{|N_k(x)|} \sum_{i: X_i \in N_k(x)} Y_i.$$

Note. k is a tuning parameter.

KNN is a **local** method: only a small number of training data close to the input are used to predict the output. As the dimension of the input increases, KNN needs exponentially more training data to stay precise.

If we make strong assumptions about the form of the regression function, **global** methods can be better than local methods. Consider a simple polynomial model for the regression function (Taylor around sample mean):

$$\begin{aligned} \mathbb{E}[Y | X = x] = f(x) &= f(\bar{x}) + f'(\bar{x})(x - \bar{x}) + \frac{1}{2}f''(\bar{x})(x - \bar{x})^2 + \dots \\ &\equiv \beta_0 + \beta_1(x - \bar{x}) + \beta_2(x - \bar{x})^2 + \dots \end{aligned}$$

That is, $f_p(x) \approx \beta_0 + \beta_1(x - \bar{x}) + \dots + \beta_p(x - \bar{x})^p$. Comparing this to KNN:

- Global: $\hat{\beta}_j$ depends on all of $X_1, \dots, X_n$.
- Local (KNN): $\hat{f}_k(x)$ is not affected by changing $X_i \notin N_k(x)$.

Generally:

- Global: better predictions when model is “correct” (low bias, low variance). For example, if the regression function is linear, then linear regression works well (zero bias, lowest possible variance).
- Local: requires a lot more data, make fewer assumptions, less sensitive to violations of assumptions, more flexible.

1.2 Model Assessment and Selection

1.2.1 Generalization Error

We want to assess the fit of a predictive model to the actual data.

Definition 1.4 (Generalization Error).

The **generalization error** of a predictive model is the expected loss of predicting a new input/output pair, conditional on having fit the model to the training data. That is, if $(X, Y) \sim P_{X,Y}$, $\mathcal{T} \sim P_{X,Y}^n$ and $\hat{f} = \mathcal{A}(\mathcal{T})$, then the generalization error is

$$\text{Err}(\mathcal{T}) = \mathbb{E}[\ell(Y^*, \hat{f}(X^*)) \mid \mathcal{T}], \quad (X^*, Y^*) \sim P_{X,Y}.$$

One way to estimate the generalization error is to perform a **train-test split**. We randomly split $\mathcal{T} = \mathcal{T}_{\text{train}} \cup \mathcal{T}_{\text{test}}$ such that $\mathcal{T}_{\text{train}} \cap \mathcal{T}_{\text{test}} = \emptyset$. Fit the model only on $\mathcal{T}_{\text{train}}$:

$$\hat{f}_{\text{train}} = \mathcal{A}(\mathcal{T}_{\text{train}}).$$

Then, we can estimate the generalization error by evaluating on the test set:

$$\widehat{\text{Err}}(\mathcal{T}) = \frac{1}{|\mathcal{T}_{\text{test}}|} \sum_{i=1}^{|\mathcal{T}_{\text{test}}|} \ell(y_i^{\text{test}}, \hat{f}_{\text{train}}(x_i^{\text{test}})).$$

This works when you have a lot of data. Ideally, the test set should be used only once at the end.

Definition 1.5 (Training Error).

The **training error** is the average loss over the training set:

$$\widehat{\text{Err}}_{\text{train}}(\mathcal{T}) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{f}(x_i))$$

where $n = |\mathcal{T}|$ and $\hat{f} = \mathcal{A}(\mathcal{T})$.

Note. Training error does not estimate the generalization error well because the model is fitted to minimize the training error. For example, you can choose $\hat{f} = y_i$ for x_i in the training set, which gives zero training error but $\text{Err}(\mathcal{T})$ is large.

1.2.2 Bias-Variance Tradeoff

Consider the following decomposition of regression function with square loss. The model is

$$Y = f(X) + \epsilon, \quad \epsilon \sim N(0, \sigma^2 I).$$

The expected loss is

$$\begin{aligned} \mathbb{E}[\ell(Y, \hat{f}(x_0)) \mid X = x_0] &= \mathbb{E}[(Y - \hat{f}(x_0))^2 \mid X = x_0] \\ &= \sigma^2 + (\mathbb{E}[\hat{f}(x_0)] - f(x_0))^2 + \mathbb{E}[(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)])^2] \\ &= \text{Irreducible Error} + \text{Bias}^2 + \text{Variance}. \end{aligned}$$

- Simple model $\implies$ high bias, low variance.
- Complex model $\implies$ low bias, high variance.

The goal is to balance bias and variance so that the generalization error is small.

1.2.3 Optimism

The training error is an optimistic estimate of the generalization error:

$$\widehat{\text{Err}}_{\text{train}}(\mathcal{T}) \leq \text{Err}(\mathcal{T}).$$

Another way to estimate the generalization error is to estimate the gap between the training error and the generalization error, and then add this to the training error.

Definition 1.6 (In-Sample Error).

The **in-sample error** is

$$\text{Err}_{\text{in}}(\mathcal{T}) = \frac{1}{n} \sum_{i=1}^n \mathbb{E} [\ell(Y_i^*, \hat{f}(x_i)) \mid \mathcal{T}]$$

where $(x_i, Y_i^*) \sim P_{X,Y}$. The inputs $x_1, \dots, x_n$ are fixed and the outputs $Y_1^*, \dots, Y_n^*$ are new outputs generated at the same input x_i .

Note. $x_i \in \mathcal{T}$ but $Y_i^* \notin \mathcal{T}$ as it is a new output generated at the same input x_i .

Definition 1.7 (Optimism, Average Optimism).

The **optimism** is the difference between the in-sample error and the training error:

$$\text{op}(\mathcal{T}) = \text{Err}_{\text{in}}(\mathcal{T}) - \widehat{\text{Err}}_{\text{train}}(\mathcal{T}).$$

The **average optimism** is the expectation of the optimism over the training set:

$$\omega = \mathbb{E} [\text{op}(\mathcal{T})].$$

Proposition 1.1. It can be shown that

$$\omega = \frac{2}{n} \sum_{i=1}^n \text{Cov}(y_i, \hat{y}_i).$$

Note. Intuition: if $\hat{y}_i$ changes a lot when y_i changes, then $\text{Cov}(y_i, \hat{y}_i)$ is large, which means the model is fitting the training data closely. So,

$$\text{large Cov}(y_i, \hat{y}_i) \implies \text{large optimism} \implies \text{training error is too small.}$$

Corollary 1.2. It follows that

$$\mathbb{E} [\text{Err}_{\text{in}}(\mathcal{F})] = \underbrace{\mathbb{E} [\widehat{\text{Err}}_{\text{train}}(\mathcal{F})]}_{\text{fit}} + \underbrace{\frac{2}{n} \sum_{i=1}^n \text{Cov}(y_i, \hat{y}_i)}_{\text{complexity}}.$$

Note. Training error measures the fit and the covariance term measures the complexity. We focus on balancing the fit and complexity. This is called the **covariance penalty** approach to model selection.

1.2.4 Degrees of Freedom

Definition 1.8 (Degrees of Freedom).

The **degrees of freedom** of a predictive model is defined as

$$\text{df}(\hat{f}; \mathcal{F}) = \frac{1}{\sigma^2} \sum_{i=1}^n \text{Cov}(y_i, \hat{f}(x_i))$$

where $\sigma^2 = \text{Var}(y_i)$ and $\mathcal{F} = \{(x_1, y_1), \dots, (x_n, y_n)\}$ for $(x_i, y_i) \stackrel{\text{iid}}{\sim} P_{X,Y}$.

Example. The above definition agrees with the one used in classical statistics (at least in linear regression). For linear regression, the model is

$$Y_i = x_i^\top \beta + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma^2)$$

where $\beta \in \mathbb{R}^p$ and $i = 1, \dots, n$. Let $X = (x_1^\top, \dots, x_n^\top)^\top$. Then,

$$\hat{\beta} = (X^\top X)^{-1} X^\top Y \quad \text{and} \quad \hat{Y} = X \hat{\beta} = X(X^\top X)^{-1} X^\top Y = HY.$$

Note that $\hat{Y}$ lies in the column space of X . If X has full rank, then the dimension of the column space is p . So the degrees of freedom is p .

Using above definition, note that

$$\begin{aligned}
\sum_{i=1}^n \text{Cov}(y_i, \hat{y}_i) &= \text{tr}(\text{Cov}(Y, HY)) \\
&= \text{tr}(H \text{Cov}(Y, Y)) \\
&= \text{tr}(H \sigma^2 I_n) \\
&= \sigma^2 \text{tr}(H) \\
&= \sigma^2 p.
\end{aligned}$$

So the degrees of freedom is $\text{df}(\hat{f}; \mathcal{T}) = \frac{1}{\sigma^2} \cdot \sigma^2 p = p$.

Note. In classical statistics, df is the dimension of the prediction space. In predictive modelling, df extends to a more general measure of model complexity.

In summary, Corollary 1.2 becomes

$$\mathbb{E}[\text{Err}_{\text{in}}(\mathcal{T})] = \mathbb{E}[\widehat{\text{Err}}_{\text{train}}(\mathcal{T})] + \frac{2\sigma^2}{n} \text{df}(\hat{f}; \mathcal{T}).$$

Example. Consider the following two linear regression models:

- Model 1: $Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \epsilon_i$.
- Model 2: $Y_i = \beta_0 + \beta_1 x_{i1} + \epsilon_i$.

Note that $\text{df}(\text{model 1}) = 3$ and $\text{df}(\text{model 2}) = 2$. Then, we compare the in-sample errors

$$\begin{aligned}
\mathbb{E}[\text{Err}_{\text{in}}(\text{model 1})] &\approx \widehat{\text{Err}}_{\text{train}}(\text{model 1}) + \frac{2\sigma^2}{n} \cdot 3 \\
\mathbb{E}[\text{Err}_{\text{in}}(\text{model 2})] &\approx \widehat{\text{Err}}_{\text{train}}(\text{model 2}) + \frac{2\sigma^2}{n} \cdot 2.
\end{aligned}$$

We choose the model with smaller in-sample error.

Example. What if we have the following two models?

- Model 1: $Y_i = \beta_0 + \beta_1 x_{i1} + \epsilon_i$.
- Model 2: $Y_i = \beta_0 + \beta_2 x_{i2} + \epsilon_i$.

The two models have the same df (complexity), so we choose the one with smaller $\text{Err}_{\text{train}}$.

1.2.5 Covariance Penalties

Recall that

$$\mathbb{E} [\text{Err}_{\text{in}}(\mathcal{J})] = \mathbb{E} [\widehat{\text{Err}}_{\text{train}}(\mathcal{J})] + \frac{2}{n} \sum_{i=1}^n \text{Cov}(y_i, \hat{y}_i).$$

The covariance penalty approach to model selection minimizes

$$\widehat{\text{Err}}_{\text{train}}(\mathcal{J}) + \frac{2}{n} \sum_{i=1}^n \text{Cov}(y_i, \hat{y}_i).$$

Two popular examples of covariance penalties are **Mallows'** C_p and **Akaike information criterion** (AIC). For linear regression (with square loss),

$$C_p = \widehat{\text{Err}}_{\text{train}}(\mathcal{J}) + \frac{2\hat{\sigma}^2}{n} \cdot p.$$

And

$$\text{AIC} = -\frac{2}{n} (\log\widehat{\text{likelihood}} + p)$$

where $\log\widehat{\text{likelihood}}$ is the maximized log-likelihood. AIC generalizes C_p . In a linear regression model with square loss, it can be shown that

$$\text{AIC} = \frac{1}{\hat{\sigma}^2} C_p.$$

Problem: both require estimating σ^2 .

This motivates to use **cross-validation** to estimate the generalization error without estimating σ^2 .

1.3 Cross-Validation

Definition 1.9 (Cross-Validation).

Cross-validation is a computational technique that provides a point estimate of the average generalization error of a predictive model by repeatedly splitting data into training/test sets, fitting the model on each training set and estimating error on the corresponding test set. Then, average the test set error estimates.

Note. The average generalization error is

$$\text{Err} = \mathbb{E}_{\mathcal{T} \sim P_{X,Y}^n} [\text{Err}(\mathcal{T})] = \mathbb{E}_{\mathcal{T} \sim P_{X,Y}^n} [\mathbb{E}[\ell(Y^*, \hat{f}(X^*)) \mid \mathcal{T}]]$$

where $\hat{f} = \mathcal{A}(\mathcal{T})$ and $(X^*, Y^*) \sim P_{X,Y}$ is a new point.

1.3.1 K-Fold Cross-Validation

The method of K -fold cross-validation is as follows. Suppose $\mathcal{T} = \{(x_1, y_1), \dots, (x_n, y_n)\}$ is the training set. First, uniformly assign each (x_i, y_i) to fold $K(i) \in \{1, \dots, K\}$ so that each fold has size $\frac{n}{K}$. For each fold j , we remove fold j and fit the model on the remaining folds:

$$\mathcal{T}_j = \{(x_i, y_i) : K(i) \neq j, i = 1, \dots, n\} \quad \text{and} \quad \hat{f}^{-j} = \mathcal{A}(\mathcal{T}_j), \quad j = 1, \dots, K.$$

Note that $|\mathcal{T}_j| = n - \frac{n}{K} = n \left(1 - \frac{1}{K}\right)$ and we fit the model K times.

For fold j , we test $\hat{f}^{-j}$ on the data that was removed so the average loss on the held-out fold is

$$L_j = \frac{1}{n/K} \sum_{i: K(i)=j} \ell(y_i, \hat{f}^{-j}(x_i))$$

where $(x_i, y_i) \in \text{fold } j$ and $\hat{f}^{-j}$ is trained on folds $1, \dots, j-1, j+1, \dots, K$. Then, the CV estimate of average generalization error is the average of hold-out losses:

$$\widehat{\text{Err}} = \frac{1}{K} \sum_{j=1}^K L_j.$$

This is also equal to the average loss of each data point, computed using the hold-out error estimates:

$$\widehat{\text{Err}} = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{f}^{-K(i)}(x_i)).$$

Example. Suppose $n = 6$ and $K = 2$. The training set is

$$\mathcal{T} = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4), (x_5, y_5), (x_6, y_6)\}.$$

The fold assignments are

$$K(1) = 1, K(2) = 2, K(3) = 1, K(4) = 1, K(5) = 2, K(6) = 2.$$

So,

$$\mathcal{T}_1 = \{(x_2, y_2), (x_5, y_5), (x_6, y_6)\} \quad \text{and} \quad \hat{f}^{-1} = \mathcal{A}(\mathcal{T}_1),$$

$$\mathcal{T}_2 = \{(x_1, y_1), (x_3, y_3), (x_4, y_4)\} \quad \text{and} \quad \hat{f}^{-2} = \mathcal{A}(\mathcal{T}_2).$$

Then, the CV estimate is

$$\widehat{\text{Err}} = \frac{1}{6} (\ell(y_1, \hat{f}^{-1}(x_1)) + \dots + \ell(y_6, \hat{f}^{-2}(x_6))).$$

Note (How to choose k ?).

If K is large ($K \approx n$):

- Each fold is small with $\frac{n}{K} \approx 1$ and training sets are of size $n - \frac{n}{K} \approx n$.
- $\widehat{\text{Err}}$ estimates $\mathbb{E} [\text{Err}(\mathcal{T})]$ on training sets of size almost n . So, low bias.
- But test sets are small, each individual test loss $\ell(y_i, \hat{f}^{-K(i)}(x_i))$ can jump around a lot, so high variance.

If K is small ($K \approx 1$):

- Each fold is large and training sets are of size $n - \frac{n}{K}$, which is small.
- $\widehat{\text{Err}}$ estimates $\mathbb{E} [\text{Err}(\mathcal{T})]$ on training sets of size much smaller than n . So, high bias.
- But test sets are large, so low variance.

If $K = n$, the method is called **leave-one-out cross-validation** (LOOCV).

1.3.2 Leave-One-Out Cross-Validation

When $K = n$, every fold has size 1. The training set is

$$\mathcal{T}_i = \mathcal{T} \setminus \{(x_i, y_i)\} \quad \text{and} \quad \hat{f}^{-i} = \mathcal{A}(\mathcal{T}_i), \quad i = 1, \dots, n.$$

The LOOCV estimate is

$$\widehat{\text{Err}} = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{f}^{-i}(x_i)).$$

This has approximately zero bias, but high variance.

Choosing Models Using CV

Suppose we fit $d \in \mathbb{N}$ models:

$$f_1 = \mathcal{A}_1(\mathcal{T}), \dots, f_d = \mathcal{A}_d(\mathcal{T})$$

and score them using CV. We choose the model with the smallest $\widehat{\text{Err}}$. We can also use this to choose tuning parameters (i.e. a value you choose before fitting the model). For example,

$$\hat{f}_\lambda = \mathcal{A}(\mathcal{T}; \lambda)$$

where λ is a tuning parameter (e.g. k in KNN). One way to choose λ is to compute

$$\widehat{\text{Err}}(\lambda) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{f}_\lambda^{-K(i)}(x_i))$$

Let $\hat{\lambda} = \arg \min_{\lambda} \widehat{\text{Err}}(\lambda)$. Then, we choose $\hat{f}_{\hat{\lambda}} = \mathcal{A}(\mathcal{T}; \hat{\lambda})$.

In models with predictions that are linear projections into $\text{Span}(X)$ i.e. $\hat{y} = Hy$ where $H = X(X^\top X)^{-1}X^\top$ is projection into $\text{Span}(X)$, for $X \in \mathbb{R}^{n \times p}$. LOOCV can be computed using a single fit of the model.

Proposition 1.3. If $\hat{y} = Hy$ where $H = X(X^\top X)^{-1}X^\top$ is the projection matrix, then

$$\widehat{\text{Err}} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_{ii}} \right)^2.$$

Note. For linear regression, fit the model only once ($\hat{f} = \mathcal{A}(\mathcal{T})$) to get $\hat{y}_i$ and h_{ii} . Then, compute the LOOCV estimate using above.

Proof. Suppose $\hat{y} = Hy$ where $H = X(X^\top X)^{-1}X^\top$ and $X \in \mathbb{R}^{n \times p}$ with $p < n$ and $\text{rank}(X) = p$. For leave-one-out, we define the fitted values using the remaining $n - 1$ observations:

$$\hat{y}^{-i} = H^{-i}y^{-i}$$

where $H^{-i} = X_{-i}((X_{-i})^\top X_{-i})^{-1}(X_{-i})^\top$ and $y^{-i} = (y_1, \dots, y_{i-1}, y_{i+1}, \dots, y_n)^\top$ and X_{-i} is X with the i -th row removed. Note that

$$X_{-i}^\top X_{-i} = X^\top X - x_i x_i^\top$$

where x_i is the i th row of X . Similarly,

$$X_{-i}^\top y_{-i} = X^\top y - x_i y_i.$$

We want to show that $y_i - \hat{f}^{-i}(x_i) = \frac{y_i - \hat{y}_i}{1 - h_{ii}}$. Note that $\hat{y} = Hy = X\hat{\beta}$ where $\hat{\beta} = (X^\top X)^{-1}X^\top y$. Then,

$$\hat{\beta}^{-i} = (X_{-i}^\top X_{-i})^{-1}X_{-i}^\top y_{-i}.$$

By the Sherman-Morrison formula,

$$\begin{aligned} (X_{-i}^\top X_{-i})^{-1} &= (X^\top X - x_i x_i^\top)^{-1} \\ &= (X^\top X)^{-1} + \frac{(X^\top X)^{-1} x_i x_i^\top (X^\top X)^{-1}}{1 - x_i^\top (X^\top X)^{-1} x_i} \\ &= (X^\top X)^{-1} + \frac{(X^\top X)^{-1} x_i x_i^\top (X^\top X)^{-1}}{1 - h_{ii}}. \end{aligned}$$

So,

$$\begin{aligned} \hat{\beta}^{-i} &= \left((X^\top X)^{-1} + \frac{(X^\top X)^{-1} x_i x_i^\top (X^\top X)^{-1}}{1 - h_{ii}} \right) (X^\top y - x_i y_i) \\ &= (X^\top X)^{-1} X^\top y + \frac{(X^\top X)^{-1} x_i x_i^\top (X^\top X)^{-1} X^\top y}{1 - h_{ii}} - (X^\top X)^{-1} x_i y_i - \frac{(X^\top X)^{-1} x_i x_i^\top (X^\top X)^{-1} x_i y_i}{1 - h_{ii}} \\ &= \hat{\beta} + \frac{(X^\top X)^{-1} x_i x_i^\top}{1 - h_{ii}} \hat{\beta} - (X^\top X)^{-1} x_i y_i - \frac{h_{ii} y_i}{1 - h_{ii}} (X^\top X)^{-1} x_i \\ &= \hat{\beta} + \frac{(X^\top X)^{-1} x_i}{1 - h_{ii}} (x_i^\top \hat{\beta} - (1 - h_{ii}) y_i - h_{ii} y_i) \\ &= \hat{\beta} + \frac{(X^\top X)^{-1} x_i}{1 - h_{ii}} (\hat{y}_i - y_i). \end{aligned}$$

Substituting this into $\hat{f}^{-i}(x_i) = x_i^\top \hat{\beta}^{-i}$ gives

$$\begin{aligned}\hat{f}^{-i}(x_i) &= x_i^\top \hat{\beta} + \frac{x_i^\top (X^\top X)^{-1} x_i}{1 - h_{ii}} (\hat{y}_i - y_i) \\ &= \hat{y}_i + \frac{h_{ii}}{1 - h_{ii}} (\hat{y}_i - y_i) \\ &= \frac{\hat{y}_i - h_{ii} y_i}{1 - h_{ii}}.\end{aligned}$$

So,

$$y_i - \hat{f}^{-i}(x_i) = y_i - \frac{\hat{y}_i - h_{ii} y_i}{1 - h_{ii}} = \frac{y_i - \hat{y}_i}{1 - h_{ii}}.$$

□

1.3.3 Generalized Cross-Validation

There are some problems with Proposition 1.3.

- If n is large, computing h_{ii} might be inefficient since H is $n \times n$.
- If some observations have high influence, then they will have a disproportionate effect on the LOOCV estimate of error.
- LOOCV is not rotationally invariant, i.e. the estimate of error can change if we apply an orthogonal transformation to the input data.

Let $Q \in \mathbb{R}^{n \times n}$ be an orthogonal matrix, i.e. $Q^\top Q = QQ^\top = I_n$. If we apply Q to the input data (i.e. $y \mapsto Qy$ and $X \mapsto QX$), then

$$(Qy - Q\hat{y})^\top (Qy - Q\hat{y}) = (y - \hat{y})^\top Q^\top Q (y - \hat{y}) = (y - \hat{y})^\top (y - \hat{y}).$$

That is, $\sum (y_i - \hat{y}_i)^2$ is unchanged by rotating y and X . However, we can show that rotated h_{ii} is not the same as h_{ii} , so $\widehat{\text{Err}}$ will change.

Definition 1.10 (Generalized CV).

The **generalized CV** (GCV) estimate of error is

$$\text{GCV} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - \frac{\text{tr}(H)}{n}} \right)^2.$$

Note. GCV is invariant to rotations, i.e. $\text{tr}(QH Q^\top) = \text{tr}(H Q Q^\top) = \text{tr}(H)$. Also, $\text{tr}(H)$ is easier to compute than h_{ii} . For least squares, $\text{tr}(H) = p$.

2 Regularization and Penalized Regression

2.1 Linear Regression via Least Squares

2.1.1 Derivation and Properties

Recall that the linear regression model is the simplest predictive modelling technique. The regression function is

$$f(x) = \mathbb{E}[Y | X = x].$$

Also,

$$f(x) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p = x^\top \beta, \quad x = (1, x_1, \dots, x_p)^\top.$$

The method of least squares is used to estimate β . For $\hat{Y} = X\hat{\beta}$,

$$\hat{\beta} = \arg \min_{\beta \in \mathbb{R}^{p+1}} \|Y - X\beta\|_2^2.$$

Under a Gaussian model, the least squares is equivalent to both the method of moments and maximum likelihood.

Maximum Likelihood

Suppose a Gaussian model: $Y_i = x_i^\top \beta + \epsilon_i$, where $\epsilon_i \stackrel{\text{iid}}{\sim} N(0, \sigma^2)$. The likelihood function is

$$\begin{aligned} L(\beta, Y; X) &= \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(Y_i - x_i^\top \beta)^2}{2\sigma^2}\right) \\ &= (2\pi\sigma^2)^{-\frac{n}{2}} \exp\left(-\frac{1}{2\sigma^2} \|Y - X\beta\|_2^2\right). \end{aligned}$$

Thus, the MLE is

$$\hat{\beta} = \arg \max_{\beta \in \mathbb{R}^{p+1}} L(\beta, Y; X) = \arg \min_{\beta \in \mathbb{R}^{p+1}} \|Y - X\beta\|_2^2.$$

Method of Moments

Match sample moments with population moments. The model says

$$\mathbb{E}[Y_i | X] = x_i^\top \beta = \sum_{j=1}^p x_{ij} \beta_j.$$

Then,

$$\mathbb{E}[Y_i x_{ij} | X] = x_{ij} \sum_{\ell=1}^p x_{i\ell} \beta_{\ell} \implies \sum_{i=1}^n Y_i x_{ij} = \sum_{i=1}^n \sum_{\ell=1}^p x_{ij} x_{i\ell} \beta_{\ell}, \quad \forall j = 1, \dots, p.$$

In matrix form,

$$X^T Y = X^T X \beta \implies X^T (Y - X \beta) = 0 \implies \hat{\beta} = \arg \min_{\beta \in \mathbb{R}^{p+1}} \|Y - X \beta\|_2^2.$$

Least squares makes predictions that are the orthogonal projection of the outputs Y onto the column space of the inputs X where

$$\text{Span}(X) = \{Y \in \mathbb{R}^n : Y = X \beta \text{ for some } \beta \in \mathbb{R}^{p+1}\}.$$

The least squares solves $\hat{Y} = \operatorname{argmin}_{a \in \text{Span}(X)} \|Y - a\|_2^2$. Note that

$$\min_{a \in \text{Span}(X)} \|Y - a\|_2^2 = \min_{\beta \in \mathbb{R}^{p+1}} \|Y - X \beta\|_2^2 \implies \hat{Y} = X \hat{\beta}.$$

Solving for $\hat{\beta}$,

$$\begin{aligned} \frac{\partial}{\partial \beta} \|Y - X \beta\|_2^2 &= \frac{\partial}{\partial \beta} (Y - X \beta)^T (Y - X \beta) \\ &= \frac{\partial}{\partial \beta} (Y^T Y - 2Y^T X \beta + \beta^T X^T X \beta) \\ &= -2X^T Y + 2X^T X \beta \stackrel{\text{set}}{=} 0. \end{aligned}$$

Then,

$$X^T Y - X^T X \hat{\beta} = 0 \implies \hat{\beta} = (X^T X)^{-1} X^T Y.$$

So, $\hat{Y} = X \hat{\beta} = X(X^T X)^{-1} X^T Y = H Y$. Geometrically, this means the least squares prediction chooses $\hat{Y}$ to be the orthogonal projection of Y onto $\text{Span}(X)$ (closest vector to Y in $\text{Span}(X)$).

2.1.2 Distributional Properties

Proposition 2.1. Under the Gaussian model $Y \sim N(X\beta, \sigma^2 I_n)$, the sampling distribution of $\hat{\beta}$ is $N(\beta, \sigma^2(X^\top X)^{-1})$.

Proof. Note that

$$\mathbb{E}[\hat{\beta}] = \mathbb{E}[(X^\top X)^{-1}X^\top Y] = (X^\top X)^{-1}X^\top \mathbb{E}[Y] = (X^\top X)^{-1}X^\top X\beta = \beta.$$

Also,

$$\begin{aligned} \text{Var}(\hat{\beta}) &= \text{Var}((X^\top X)^{-1}X^\top Y) \\ &= (X^\top X)^{-1}X^\top \text{Var}(Y)X[(X^\top X)^{-1}]^\top \quad \text{since } \text{Var}(AY) = A \text{Var}(Y)A^\top \end{aligned}$$

Note that $[(X^\top X)^{-1}]^\top = [(X^\top X)^\top]^{-1} = (X^\top X)^{-1}$.

$$\begin{aligned} &= (X^\top X)^{-1}X^\top (\sigma^2 I)X(X^\top X)^{-1} \\ &= \sigma^2 (X^\top X)^{-1}X^\top X(X^\top X)^{-1} \\ &= \sigma^2 (X^\top X)^{-1}. \end{aligned}$$

□

Note. $\forall a \in \mathbb{R}^p, \mathbb{E}[a^\top \hat{\beta}] = a^\top \beta$.

Theorem 2.2 (Gauss-Markov Theorem).

The least squares estimator $\hat{\beta}$ has the smallest variance among all linear unbiased estimators of β .

Proof. Suppose $\tilde{\beta} = CY$ where $C \in \mathbb{R}^{p \times n}$. Write $C = (X^\top X)^{-1}X^\top + D$ for some $D \in \mathbb{R}^{p \times n}$. Assume that $\tilde{\beta}$ is unbiased, i.e. $\mathbb{E}[\tilde{\beta}] = \mathbb{E}[\hat{\beta}] = \beta$. Then,

$$\begin{aligned} \mathbb{E}[\tilde{\beta}] &= \mathbb{E}[CY] = C\mathbb{E}[Y] = CX\beta \\ &= (X^\top X)^{-1}X^\top X\beta + DX\beta = \beta + DX\beta. \end{aligned}$$

So, $\tilde{\beta}$ is unbiased $\iff DX = 0$. For variance,

$$\begin{aligned}\text{Var}(\tilde{\beta}) &= \text{Var}(CY) = C \text{Var}(Y) C^\top = C(\sigma^2 I) C^\top = \sigma^2 C C^\top \\ &= \sigma^2 [(X^\top X)^{-1} X^\top + D] [(X^\top X)^{-1} X^\top + D]^\top \\ &= \sigma^2 [(X^\top X)^{-1} + D D^\top] \quad \text{since } DX = 0 \\ &= \sigma^2 (X^\top X)^{-1} + \sigma^2 \underbrace{D D^\top}_{\geq 0} \geq \sigma^2 (X^\top X)^{-1} = \text{Var}(\hat{\beta}).\end{aligned}$$

Here, $A \geq B$ means $A - B$ is positive semidefinite. So if $a \in \mathbb{R}^p$, then

$$\begin{aligned}\text{Var}(a^\top \tilde{\beta}) &= a^\top \text{Var}(\tilde{\beta}) a = \sigma^2 a^\top (X^\top X)^{-1} a + \sigma^2 a^\top D D^\top a \\ &= \text{Var}(a^\top \hat{\beta}) + \|\sigma D^\top a\|_2^2 \geq \text{Var}(a^\top \hat{\beta}).\end{aligned}$$

So, $\hat{Y} = X\hat{\beta}$ has the smallest variance. □

Note. X is assumed to have full rank, i.e. $(X^\top X)^{-1}$ exists. If X does not have full rank, then there are zero or infinitely many least squares solutions, in which case any one of them can be used to predict.

2.1.3 Spectral Analysis

Consider the **singular value decomposition** (SVD) of $X \in \mathbb{R}^{n \times p}$ ($p < n$):

$$X = U D V^\top$$

where

- $U \in \mathbb{R}^{n \times p}$, the columns of U form an orthonormal basis for $\text{Span}(X)$, i.e. $U^\top U = I_p$.
- $V \in \mathbb{R}^{p \times p}$ is an orthogonal matrix, i.e. $V^\top V = V V^\top = I_p$.
- $D = \text{diag}(\delta_1, \dots, \delta_p)$ with $\delta_1 \leq \dots \leq \delta_p$ being the singular values of X .

Note that

$$X^\top X = (V D U^\top)(U D V^\top) = V D^2 V^\top$$

and $D^2 = \text{diag}(\delta_1^2, \dots, \delta_p^2)$ where δ_j^2 is the j -th eigenvalue of $X^\top X$ (we assume $\delta_j^2 > 0$, i.e. X has full

rank). Then,

$$\begin{aligned}
\hat{Y} &= X(X^\top X)^{-1}X^\top Y \\
&= UDV^\top(VD^2V^\top)^{-1}VDU^\top Y \\
&= UDV^\top(V^\top)^{-1}D^{-1}D^{-1}V^{-1}VDU^\top Y \\
&= UDD^{-1}D^{-1}DU^\top Y = UU^\top Y.
\end{aligned}$$

Proposition 2.3. The least squares prediction can be written as $\hat{Y} = UU^\top Y$.

If we add more highly correlated inputs, then the smallest singular value δ_1 of X will be close to 0, indicating two or more inputs are nearly linearly dependent ($\delta_j = 0 \implies X$ not full rank so $X^\top X$ not invertible). So, $\hat{\beta}$ cannot be uniquely determined. The prediction can still be made, but $\text{Var}(\hat{\beta})$ will be very large! Note that

$$\begin{aligned}
\text{Var}(\hat{Y}) &= \text{Var}(UU^\top Y) = UU^\top \text{Var}(Y) UU^\top \\
&= \sigma^2 UU^\top UU^\top = \sigma^2 UU^\top.
\end{aligned}$$

Since $\hat{Y} = X\hat{\beta}$,

$$\text{Var}(\hat{\beta}) = \sigma^2(X^\top X)^{-1} = \sigma^2 VD^{-2}V^\top, \quad \text{where } D^{-2} = \text{diag}(\delta_1^{-2}, \dots, \delta_p^{-2}).$$

As $\delta_1 \rightarrow 0$, $\frac{1}{\delta_1^2} \rightarrow \infty$, so $\text{Var}(\hat{\beta}) \rightarrow \infty$.

Note. This motivates ridge regression.

2.2 Ridge Regression

When two or more columns of X are linearly dependent, the variance of the predictions is infinite. Ridge regression fixes this instability problem.

2.2.1 Spectral Motivation

Consider the least squares predictions:

$$\hat{Y} = X(X^\top X)^{-1}X^\top Y.$$

If some singular values of X are close to 0 (i.e. some $\delta_j^2 \approx 0$), then $(X^\top X)^{-1}$ can be unstable. Ridge regression adds a small value $\lambda > 0$ to the diagonal of $X^\top X$ to make it invertible:

$$\hat{Y}(\lambda) = X(X^\top X + \lambda I_p)^{-1} X^\top Y \equiv H(\lambda)Y.$$

From the SVD of X , we have $X^\top X = VD^2V^\top$ and $VV^\top = V^\top V = I_p$. Now, ridge uses

$$X^\top X + \lambda I_p = VD^2V^\top + \lambda VV^\top = V(D^2 + \lambda I_p)V^\top.$$

So, the eigenvalues of $X^\top X + \lambda I_p$ are $\delta_j^2 + \lambda > 0$, so $X^\top X + \lambda I_p$ is invertible.

The ridge regression predictions now satisfy

$$\begin{aligned} \hat{Y}(\lambda) &= X(X^\top X + \lambda I_p)^{-1} X^\top Y \\ &= UDV^\top (V(D^2 + \lambda I_p)V^\top)^{-1} VDU^\top Y \\ &= UDV^\top (V^\top)^{-1} (D^2 + \lambda I_p)^{-1} V^{-1} VDU^\top Y \\ &= UD(D^2 + \lambda I_p)^{-1} DU^\top Y \\ &= U \operatorname{diag} \left(\frac{\delta_1^2}{\delta_1^2 + \lambda}, \dots, \frac{\delta_p^2}{\delta_p^2 + \lambda} \right) U^\top Y \\ &= \sum_{j=1}^p u_j u_j^\top \frac{\delta_j^2}{\delta_j^2 + \lambda} Y. \end{aligned}$$

For the least squares, $\lambda = 0$, so

$$\hat{Y}(0) = \sum_{j=1}^p u_j u_j^\top Y = UU^\top Y.$$

Each input dimension $j = 1, \dots, p$ is shrunk by a factor of $\frac{\delta_j^2}{\delta_j^2 + \lambda} \in [0, 1)$. If $\delta_j^2 \gg \lambda$, then $\frac{\delta_j^2}{\delta_j^2 + \lambda} \approx 1$, so the j -th input is not shrunk much. If $\delta_j^2 \ll \lambda$, then $\frac{\delta_j^2}{\delta_j^2 + \lambda} \approx 0$, so the j -th input is shrunk a lot. So, ridge regression helps with collinearity.

- Inputs with large singular values have their influence on the predictions reduced less (i.e. $\frac{\delta_j^2}{\delta_j^2 + \lambda} \approx 1$).
- Inputs with smaller singular values have smaller influence on the predictions.
- Inputs with singular values equal to 0 have no influence on the predictions.

2.2.2 Penalized Least Squares

Consider the following constrained minimization problem:

$$\hat{\beta}(\lambda) = \underset{\beta}{\operatorname{argmin}} \|Y - X\beta\|_2^2 \quad \text{subject to } \|\beta\|_2^2 \leq t^2 \text{ for some } t > 0.$$

This is equivalent to the following unconstrained minimization problem:

$$\hat{\beta}(\lambda) = \underset{\beta}{\operatorname{argmin}} \|Y - X\beta\|_2^2 + \lambda \|\beta\|_2^2$$

Here, $\lambda > 0$ is called a **regularization parameter** or **shrinkage parameter**. Larger values of λ correspond to more shrinkage of the regression coefficients towards 0. Write

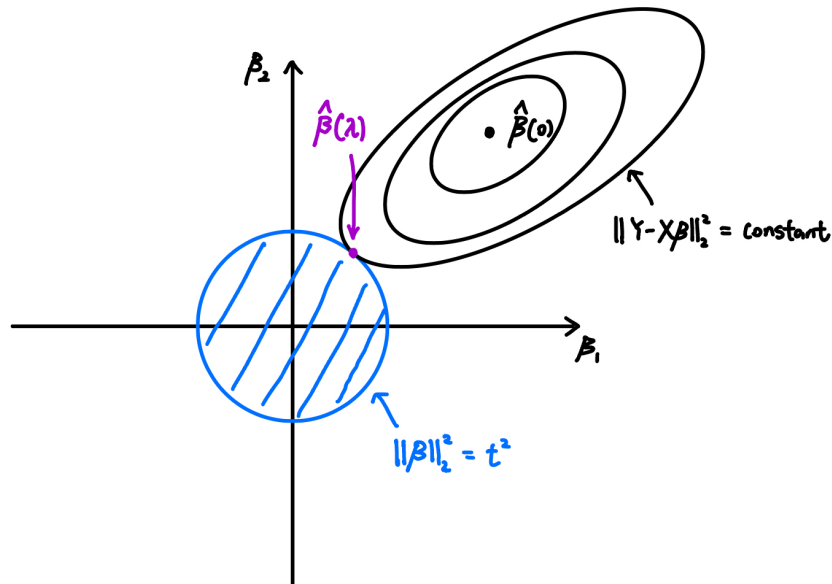
$$\ell(\beta; \lambda) \equiv \|Y - X\beta\|_2^2 + \lambda \|\beta\|_2^2.$$

The solution can be derived by setting the gradient to 0:

$$\begin{aligned} \frac{\partial}{\partial \beta} \ell(\beta; \lambda) &= \frac{\partial}{\partial \beta} (Y - X\beta)^\top (Y - X\beta) + \lambda \frac{\partial}{\partial \beta} \beta^\top \beta \\ &= -2X^\top Y + 2X^\top X\beta + 2\lambda\beta \stackrel{\text{set}}{=} 0 \end{aligned}$$

$$\Rightarrow X^\top Y = (X^\top X + \lambda I_p) \beta$$

$$\Rightarrow \hat{\beta}(\lambda) = (X^\top X + \lambda I_p)^{-1} X^\top Y.$$



Explanation of the picture: The black dot represents the least squares solution $\hat{\beta}(0)$. The blue circle represents the constraint $\|\beta\|_2^2 \leq t^2$. The purple dot represents the ridge regression solution $\hat{\beta}(\lambda)$, which is the point on the blue circle that is closest to the black dot. The black ellipses represent the contours of the residual sum of squares $\|Y - X\beta\|_2^2$. Note that the least squares solution $\hat{\beta}(0)$ is farther away from the origin than the ridge regression solution $\hat{\beta}(\lambda)$. So ridge shrinks the regression coefficients towards 0 (more shrinkage occurs in the larger direction).

Note. Tradeoff: ridge regression gives up a little fit by moving away from $\hat{\beta}(0)$ to gain stability by keeping the coefficient vector small.

Proposition 2.4 (Sampling Distribution of $\hat{\beta}(\lambda)$).

Under the Gaussian model $Y \sim N(X\beta, \sigma^2 I_n)$, we have

- (1) $\mathbb{E}[\hat{\beta}(\lambda)] = (X^\top X + \lambda I_p)^{-1} X^\top X \beta := M(\lambda) \beta$.
- (2) $\hat{\beta}(\lambda)$ is biased, i.e. $\mathbb{E}[\hat{\beta}(\lambda)] \neq \beta$.
- (3) $\text{Var}(\hat{\beta}(\lambda)) = \sigma^2 (X^\top X + \lambda I_p)^{-1} X^\top X (X^\top X + \lambda I_p)^{-1}$.

Proof.

- (1) Define $\hat{\beta}(\lambda) = M(\lambda)Y$ where $M(\lambda) = (X^\top X + \lambda I_p)^{-1} X^\top$. Then,

$$\mathbb{E}[\hat{\beta}(\lambda)] = M(\lambda) \mathbb{E}[Y] = M(\lambda) X \beta = (X^\top X + \lambda I_p)^{-1} X^\top X \beta.$$

- (2) Note that

$$\begin{aligned} \text{Bias}(\hat{\beta}(\lambda)) &= \mathbb{E}[\hat{\beta}(\lambda)] - \beta \\ &= (X^\top X + \lambda I_p)^{-1} X^\top X \beta - \beta \\ &= [(X^\top X + \lambda I_p)^{-1} X^\top X - I_p] \beta \neq 0. \end{aligned}$$

- (3) Note that

$$\begin{aligned} \text{Var}(\hat{\beta}(\lambda)) &= \text{Var}(M(\lambda)Y) = M(\lambda) \text{Var}(Y) M(\lambda)^\top \\ &= M(\lambda) (\sigma^2 I_n) M(\lambda)^\top = \sigma^2 M(\lambda) M(\lambda)^\top \\ &= \sigma^2 (X^\top X + \lambda I_p)^{-1} X^\top X (X^\top X + \lambda I_p)^{-1}. \end{aligned} \quad \square$$

Note. $\lim_{\lambda \rightarrow \infty} \text{Var}(\hat{\beta}(\lambda)) = 0 \in \mathbb{R}^{p \times p}$ since $\lim_{\lambda \rightarrow \infty} \hat{\beta}(\lambda) = 0$ with probability 1.

Proposition 2.5. The estimate of σ^2 is

$$\hat{\sigma}^2 = \frac{\|Y - X\hat{\beta}(\lambda)\|_2^2}{n - \text{tr}(H(\lambda))}$$

where $H(\lambda) = X(X^\top X + \lambda I_p)^{-1}X^\top$.

2.2.3 Penalty Parameter Selection by Generalized CV

To select the tuning parameter λ by GCV, we parameterize the GCV error as a function of λ :

$$\hat{\mathcal{E}}(\lambda) = \frac{1}{n} \sum_{i=1}^n \left(\frac{Y_i - \hat{Y}_i(\lambda)}{1 - \frac{\text{tr}(H(\lambda))}{n}} \right)^2$$

Then, we choose the λ that minimizes $\hat{\mathcal{E}}(\lambda)$:

$$\hat{\lambda} = \underset{\lambda > 0}{\operatorname{argmin}} \hat{\mathcal{E}}(\lambda).$$

Note. GCV is a practical approximation instead of a good estimate of the generalization error. Also, GCV score is computationally intensive, i.e. $H(\lambda) \in \mathbb{R}^{n \times n}$ requires $O(n^2)$ memory. For ridge regression, we can simplify the computation as follows:

$$\begin{aligned} \text{tr}(H(\lambda)) &= \text{tr}(X(X^\top X + \lambda I_p)^{-1}X^\top) \\ &= \text{tr}\left(\underbrace{(X^\top X + \lambda I_p)^{-1}X^\top X}_{p \times p}\right) \quad \text{since } \text{tr}(AB) = \text{tr}(BA). \end{aligned}$$

Then, we work with a smaller $p \times p$ matrix.

Note (Midterm 1 Info).

- Some proofs are important: Gauss-Markov theorem (more on thin Monday)
- KNN by hand (A1): need to know how KNN works, including how to find distances... Continuous, scaling, and multidimensional cases (A1 has categorical and numerical cases).
- KNN of large datasets: might ask something like A1Q2 (b), i.e. examining a graph or local vs. global methods.
- Tradeoff and the usefulness of cross-validation.
- Different loss functions.
- Degrees of freedom.

Note (More Midterm 1 Info).

- Sampling distributions
- Difference between CV and GCV
- Find dfs.
- Formula for $\hat{\beta}(\lambda)$ in ridge regression and the picture on Page 25.
- More focus on Chapter 2.

3 Nonlinear Regression

In this chapter, we extend the linear regression model to model regression functions that depend on the inputs in a nonlinear way. The key idea is to derive a set of transformed inputs that is rich and flexible and then control estimation of the coefficients via regularization (e.g. ridge regression).

3.1 Function Estimation

Consider the regression model:

$$Y_i = f(X_i) + \epsilon_i, \quad \epsilon_i \stackrel{iid}{\sim} N(0, \sigma^2)$$

where $f : \mathbb{R} \rightarrow \mathbb{R}$ is any function. Equivalently, $Y_i \stackrel{ind}{\sim} N(f(X_i), \sigma^2)$.

3.1.1 Estimating Functions is Impossible

Question: Can we uniquely and efficiently estimate f using only finitely-many $n < \infty$ data points?

Answer: No, not without additional restrictions. To see why, consider the usual least squares objective:

$$\ell(f) = \sum_{i=1}^n (Y_i - f(X_i))^2 \geq 0.$$

If we choose f such that $f(X_i) = Y_i$ for all i , then $\ell(f) = 0$. Hence, there are infinitely many such functions. But such functions can have arbitrarily high expected error on new data points.

One solution is to restrict the form of f . An example is linear regression, where f is restricted to be a linear function of the inputs. Another example is to use polynomial regression, which is still linear regression:

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_p x^p.$$

More generally,

$$f(x) = \sum_{j=0}^p h_j(x) \beta_j$$

where $x \in \mathbb{R}$ and $h_j : \mathbb{R} \rightarrow \mathbb{R}$ are any arbitrary functions of x . This is called a **linear basis function model**.

Question: Is there an optimal way to interpolate a set of n data points?

Answer: Yes. One way is the **minimum norm interpolation problem**. We choose $\hat{f} \in \arg \min_f \|f\|_{\mathcal{H}}$

subject to $f(x_i) = y_i$ for all $i = 1, \dots, n$, where $\|\cdot\|_{\mathcal{H}}$ is a norm on some space $\mathcal{H}$.

If we restrict $\mathcal{H}$ appropriately, then the solution $\hat{f}$ is unique and finite-dimensional. Note that the Lagrange form of this problem is the infinite-dimensional ridge regression problem:

$$\hat{f} \in \arg \min_f \ell(f; \lambda), \quad \ell(f) = \sum_{i=1}^n (Y_i - f(X_i))^2 + \lambda \|f\|_{\mathcal{H}}^2.$$

3.1.2 Reproducing Kernel Hilbert Spaces (RKHS)

We will introduce a type of infinite-dimensional vector space called a **Reproducing Kernel Hilbert Space (RKHS)** in which the solution to the minimum norm interpolation/infinite-dimensional ridge regression problem is unique and finite-dimensional. Then, we will show that the “Sobolev” spaces of functions with some number of absolutely continuous derivatives are RKHSs, hence, proving that we can estimate functions from those spaces using only $n < \infty$ data. This argument forms the theoretical basis for nonlinear regression by penalized least squares, also known as “drawing squiggly lines through data”.

Definition 3.1 (Hilbert Space).

A **Hilbert space** is a complete inner product space. That is, $(\mathcal{H}, \langle \cdot, \cdot \rangle)$, where $\mathcal{H}$ is a set and $\langle \cdot, \cdot \rangle : \mathcal{H} \times \mathcal{H} \rightarrow \mathbb{R}$ is an inner product.

Remark. The inner product is **bilinear** if $\forall f, g \in \mathcal{H}, \forall a, b \in \mathbb{R}, \langle af, bg \rangle = ab \langle f, g \rangle$.

Example. Let $\mathcal{H} = \mathbb{R}^d$. Then, $\mathcal{H}$ is a Hilbert space with the dot product: $\langle x, y \rangle = x \cdot y = x^\top y$. This is also bilinear. Note that

$$\begin{aligned} \langle ax, by \rangle &= ax_1by_1 + \dots + ax_dby_d \\ &= ab(x_1y_1 + \dots + x_dy_d) = ab \langle x, y \rangle. \end{aligned}$$

Definition 3.2 (Linear Functional).

A **linear functional** is a linear map from a Hilbert space to its scalar field (e.g. $\mathbb{R}$ in STAT

444). That is, $L : \mathcal{H} \rightarrow \mathbb{R}$ such that

$$L(af + g) = aL(f) + L(g), \quad \forall f, g \in \mathcal{H}, a \in \mathbb{R}.$$

Example. Function evaluation is a linear functional. For any $x \in \mathbb{R}$, define $L_x : \mathcal{H} \rightarrow \mathbb{R}$ by $L_x(f) = f(x)$. Then, $\forall f, g \in \mathcal{H}, \forall a \in \mathbb{R}$,

$$L_x(af + g) = (af + g)(x) = af(x) + g(x) = aL_x(f) + L_x(g).$$

Definition 3.3 ((Uniformly) Bounded Linear Functional).

A linear functional $L : \mathcal{H} \rightarrow \mathbb{R}$ is **(uniformly) bounded** if $\exists M > 0$ such that $|L(f)| \leq M\|f\|_{\mathcal{H}}$ for all $f \in \mathcal{H}$.

Remark. $\|f\|_{\mathcal{H}}^2 = \langle f, f \rangle$ is called the **Hilbert space norm**.

Example. In $\mathbb{R}^d$, $\|x\|^2 = \langle x, x \rangle = x^\top x$.

Theorem 3.1 (Riesz Representation Theorem).

Let L be a uniformly bounded linear functional on a Hilbert space $\mathcal{H}$. Then, $\exists$ a unique $g \in \mathcal{H}$ such that $L(f) = \langle f, g \rangle$ for all $f \in \mathcal{H}$.

Note. g is the “representer” of L .

Note. Function evaluation is also uniformly bounded. Hence, $\exists g_x \in \mathcal{H}$ such that $f(x) = L_x(f) = \langle f, g_x \rangle_{\mathcal{H}}$ for all $f \in \mathcal{H}$. Thus, function evaluation $\iff$ inner product evaluation.

Definition 3.4 (Reproducing Kernel Hilbert Spaces (RKSH)).

A Hilbert space on which the evaluation functional is bounded is called a **Reproducing Kernel Hilbert Space (RKHS)**.

Definition 3.5 (Kernel).

Let $\mathcal{X}$ be some space. A **kernel** on $\mathcal{X}$ is a function $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$.

Definition 3.6 (Positive Semi-Definite Kernel).

A kernel K is **positive semi-definite** if it is symmetric, i.e. $K(x, y) = K(y, x)$ for all $x, y \in \mathcal{X}$, and $\forall \{x_i\}_{i=1}^n \subset \mathcal{X}$, the matrix $\mathbf{K} \in \mathbb{R}^{n \times n}$ defined by $\mathbf{K}_{ij} = K(x_i, x_j)$ is positive semi-definite.

Remark. A symmetric matrix B is **positive semi-definite** if $\forall a \in \mathbb{R}^n$, $a^\top B a \geq 0$.

Proposition 3.2 (Reproducing Property).

Every RKHS has a unique positive semi-definite kernel K such that $\forall x \in \mathcal{X}$, let $g_x(y) = K(x, y)$, then $f(x) = \langle f, g_x \rangle = \langle f, K(\cdot, x) \rangle$ for all $f \in \mathcal{H}$.

Remark. The converse is also true: every positive semi-definite kernel K uniquely defines an RKHS.

Example. Consider the linear kernel $K(x, z) = \langle x, z \rangle$. Then, for any $\{x_i\}_{i=1}^n \subset \mathbb{R}^d$, define $K_{ij} = K(x_i, x_j) = \langle x_i, x_j \rangle$. Then, $\forall \alpha \in \mathbb{R}^n$,

$$\begin{aligned} \alpha^\top K \alpha &= \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j K(x_i, x_j) = \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \langle x_i, x_j \rangle \\ &= \sum_{i=1}^n \sum_{j=1}^n \langle \alpha_i x_i, \alpha_j x_j \rangle \\ &= \left\langle \sum_{i=1}^n \alpha_i x_i, \sum_{j=1}^n \alpha_j x_j \right\rangle = \left\| \sum_{i=1}^n \alpha_i x_i \right\|^2 \geq 0. \end{aligned}$$

Thus, K is a positive semi-definite kernel. Now, consider a linear function of the form

$$f_\beta(x) = \sum_{i=1}^d \beta_i x_i, \quad \text{for some } \beta \in \mathbb{R}^d.$$

Write $f_\beta(x) = \langle x, \beta \rangle$. Let $\mathcal{H}$ be the space of all such functions with the inner product $\langle f_\beta, f_\gamma \rangle = \langle \beta, \gamma \rangle$. Our kernel is the linear kernel $K(\beta, x) = \langle x, \beta \rangle = f_\beta(x)$. Then,

$$\langle f_\beta, K(\cdot, x) \rangle = \langle f_\beta, f_x \rangle = \langle \beta, x \rangle = f_\beta(x),$$

which explicitly shows the reproducing property.

Remark (Summary).

$\mathcal{H}$ is a RKHS $\iff \exists$ a unique positive semi-definite kernel K such that $f(x) = \langle f, K(\cdot, x) \rangle$ for all $f \in \mathcal{H}, x \in \mathcal{X}$.

Example (Approximating a Function using RKHS).

Suppose we have $n = 3$ data points $\{(x_1, y_1), (x_2, y_2), (x_3, y_3)\} = \{(1, 2), (2, 3), (3, 2.5)\}$ and we want to approximate a function that fits these points using the Gaussian kernel

$$K(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right).$$

Let's assume $\sigma = 1$ for simplicity. We compute the kernel matrix $\mathbf{K}$ for $\{x_1, x_2, x_3\}$:

$$\begin{aligned} \mathbf{K} &= \begin{bmatrix} K(x_1, x_1) & K(x_1, x_2) & K(x_1, x_3) \\ K(x_2, x_1) & K(x_2, x_2) & K(x_2, x_3) \\ K(x_3, x_1) & K(x_3, x_2) & K(x_3, x_3) \end{bmatrix} = \begin{bmatrix} K(1, 1) & K(1, 2) & K(1, 3) \\ K(2, 1) & K(2, 2) & K(2, 3) \\ K(3, 1) & K(3, 2) & K(3, 3) \end{bmatrix} \\ &= \begin{bmatrix} 1 & e^{-0.5} & e^{-2} \\ e^{-0.5} & 1 & e^{-0.5} \\ e^{-2} & e^{-0.5} & 1 \end{bmatrix}. \end{aligned}$$

Now, we want to find coefficients $\alpha_1, \alpha_2, \alpha_3$ such that our approximating function is

$$\hat{f}(x) = \alpha_1 K(x, x_1) + \alpha_2 K(x, x_2) + \alpha_3 K(x, x_3)$$

where $\hat{f}(x_i) = y_i$ for $i = 1, 2, 3$. This leads to $\mathbf{K}\alpha = y$:

$$\begin{bmatrix} 1 & e^{-0.5} & e^{-2} \\ e^{-0.5} & 1 & e^{-0.5} \\ e^{-2} & e^{-0.5} & 1 \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 2.5 \end{bmatrix} \implies \alpha \approx \begin{bmatrix} 0.83 \\ 2.31 \\ 0.86 \end{bmatrix}.$$

To predict $\hat{f}(x)$ for a new input $x = 1.5$, we compute:

$$\hat{f}(1.5) = 0.83K(1.5, 1) + 2.31K(1.5, 2) + 0.86K(1.5, 3) \approx 3.05.$$

3.1.3 Minimal Norm Interpolation

Problem: choose $\hat{f} \in \arg \min_{f \in \mathcal{H}} \|f\|_{\mathcal{H}}$ subject to $f(x_i) = y_i$ for all $i = 1, \dots, n$ for a chosen RKHS $\mathcal{H}$ and a sequence $\{(x_i, y_i)\}_{i=1}^n$.

Solution: Let K be the unique positive semi-definite (reproducing) kernel on $\mathcal{H}$ and $\mathbf{K} \in \mathbb{R}^{n \times n}$ be the corresponding kernel matrix with $\mathbf{K}_{ij} = K(x_i, x_j)$. Then,

- The problem is feasible (i.e. $\exists$ at least one function in $\mathcal{H}$ that can interpolate the data) if $y \in \text{Range}(K)$, i.e. $\exists \hat{\alpha} \in \mathbb{R}^n$ such that $\mathbf{K}\hat{\alpha} = y$.
- If such a feasible solution exists, the unique optimal solution is

$$\hat{f}(\cdot) = \sum_{i=1}^n \hat{\alpha}_i K(\cdot, x_i), \quad \text{where } \mathbf{K}\hat{\alpha} = y.$$

Proof. We prove the case for $f \in \mathcal{H}$ such that $f : \mathbb{R}^d \rightarrow \mathbb{R}$, i.e. $\mathcal{X} = \mathbb{R}^d$. For any $\alpha \in \mathbb{R}^n$, let $f_\alpha(\cdot) = \sum_{i=1}^n \alpha_i K(\cdot, x_i)$. Let $\mathcal{F} = \{f_\alpha : \alpha \in \mathbb{R}^n\} \subset \mathcal{H}$, where $\mathcal{F}$ is finite-dimensional. Note that

$$f_\alpha(x_j) = \sum_{i=1}^n \alpha_i K(x_j, x_i) = (\mathbf{K}\alpha)_j.$$

Hence, $f_\alpha(x_j) = y_j \iff \mathbf{K}\alpha = y$, so the problem is feasible $\iff y \in \text{Range}(K)$.

Since $\mathcal{F}$ is finite-dimensional, it is closed, and any $f \in \mathcal{H}$ can be decomposed as $f = f_\alpha + f_\perp$, where $f_\alpha \in \mathcal{F}$ and $\langle f_\perp, f_\beta \rangle = 0$ for all $f_\beta \in \mathcal{F}$. By the reproducing property,

$$\begin{aligned} f(x_j) &= \langle f, K(\cdot, x_j) \rangle \\ &= \langle f_\alpha + f_\perp, K(\cdot, x_j) \rangle \\ &= \langle f_\alpha, K(\cdot, x_j) \rangle + \underbrace{\langle f_\perp, K(\cdot, x_j) \rangle}_{=0}. \end{aligned}$$

The second term is zero because $K(\cdot, x_j) = f_{e_j} \in \mathcal{F}$, where $e_j = (0, \dots, 0, \underset{j}{1}, 0, \dots, 0)^\top \in \mathbb{R}^n$. Also, $\langle f_\alpha, K(\cdot, x_j) \rangle = f_\alpha(x_j)$ by the reproducing property. Hence, $f_\perp$ has no impact on the interpolation. Finally,

$$\begin{aligned} \|f\|_{\mathcal{H}}^2 &= \|f_\alpha + f_\perp\|_{\mathcal{H}}^2 = \|f_\alpha\|_{\mathcal{H}}^2 + \|f_\perp\|_{\mathcal{H}}^2 \\ &\geq \|f_\alpha\|_{\mathcal{H}}^2. \end{aligned}$$

Since $f_{\perp}$ has no impact on the interpolation, we can set $f_{\perp} = 0$ to minimize the norm. Hence, the unique optimal solution is $\hat{f} = f_{\hat{\alpha}}$ for some $\hat{\alpha} \in \mathbb{R}^n$ such that $\mathbf{K}\hat{\alpha} = \mathbf{y}$. $\square$

3.1.4 Infinite-Dimensional Ridge Regression

Now, we can state and solve the infinite-dimensional ridge regression problem. The model is

$$Y_i = f(X_i) + \epsilon_i, \quad f \in \mathcal{H} \text{ for some RKHS } \mathcal{H}.$$

The penalized least squares problem is

$$\hat{f} \in \arg \min_{f \in \mathcal{H}} \left\{ \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2 \right\}.$$

Solution: The solution is finite-dimensional:

$$\hat{f} = \sum_{i=1}^n \alpha_i K(\cdot, x_i)$$

for some $\alpha \in \mathbb{R}^n$ where K is the unique positive semi-definite kernel on $\mathcal{H}$. So,

$$(f(x_1), \dots, f(x_n))^{\top} = \mathbf{K}\alpha.$$

And,

$$\|f\|_{\mathcal{H}}^2 = \langle f, f \rangle_{\mathcal{H}} = \left\langle \sum_{i=1}^n \alpha_i K(\cdot, x_i), \sum_{j=1}^n \alpha_j K(\cdot, x_j) \right\rangle_{\mathcal{H}} = \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j K(x_i, x_j) = \alpha^{\top} \mathbf{K} \alpha.$$

So the objective is $\ell(\alpha) = \|\mathbf{y} - \mathbf{K}\alpha\|_2^2 + \lambda \alpha^{\top} \mathbf{K} \alpha$. Now, the problem becomes

$$\hat{\alpha} \in \arg \min_{\alpha \in \mathbb{R}^n} \ell(\alpha) = \arg \min_{\alpha \in \mathbb{R}^n} \left\{ \|\mathbf{y} - \mathbf{K}\alpha\|_2^2 + \lambda \alpha^{\top} \mathbf{K} \alpha \right\}.$$

We differentiate $\ell(\alpha)$ with respect to α and set it to zero:

$$\begin{aligned}
\frac{\partial \ell(\alpha)}{\partial \alpha} &= -2\mathbf{K}(y - \mathbf{K}\alpha) + 2\lambda\mathbf{K}\alpha = 0 \\
\implies -2\mathbf{K}y + 2\mathbf{K}^2\alpha + 2\lambda\mathbf{K}\alpha &= 0 \\
\implies (\mathbf{K}^2 + \lambda\mathbf{K})\alpha &= \mathbf{K}y \\
\implies \mathbf{K}(\mathbf{K} + \lambda I_n)\alpha &= \mathbf{K}y \\
\implies (\mathbf{K} + \lambda I_n)\alpha &= y \\
\implies \hat{\alpha} &= (\mathbf{K} + \lambda I_n)^{-1}y
\end{aligned}$$

where in the last step we assume $\mathbf{K} + \lambda I_n$ is invertible. The fitted function is

$$\hat{f}(x) = \sum_{i=1}^n \hat{\alpha}_i K(x, x_i).$$

3.1.5 Smoothing Splines: Theoretical Construction

We saw that we can estimate f in the regression model $Y_i = f(X_i) + \epsilon_i$ as long as we restrict f to be in a RKHS and appropriately penalize its size. Now, it remains to choose a RKHS that contains functions that are “plausible”. We consider the following RKHS:

Definition 3.7 (Second-Order Sobolev Space).

The **second-order Sobolev space** is the set (WLOG, consider functions on $[0, 1]$)

$$S^2[0, 1] = \left\{ f : [0, 1] \rightarrow \mathbb{R} \mid f'' \text{ exists a.e., } f(0) = f'(0) = 0, f'(x) = \int_0^x f''(t) dt \right\}.$$

Note. a.e. = almost everywhere.

Define the inner product on $S^2[0, 1]$ as

$$\langle f, g \rangle = \int_0^1 f''(z)g''(z) dz.$$

So, $\|f\|^2 = \langle f, f \rangle = \int_0^1 (f''(x))^2 dx$.

Proposition 3.3. $S^2[0, 1]$ is a RKHS with kernel:

$$K(x, y) = \int_0^1 (x - z)_+(y - z)_+ dz, \quad \text{where } a_+ = \max\{a, 0\}.$$

Observe that $\frac{d^2}{dy^2} K(y, x) = (x - y)_+$.

Proof (Sketch). We will only verify the reproducing property. Let $f \in S^2[0, 1]$. Consider its second-order Taylor expansion at 0:

$$f(x) = f(0) + f'(0)x + \int_0^1 f''(z)(x - z) dz = \int_0^1 f''(z)(x - z)_+ dz.$$

Then,

$$\langle f, K(\cdot, x) \rangle = \int_0^1 f''(z) \frac{d^2}{dz^2} K(z, x) dz = \int_0^1 f''(z)(x - z)_+ dz = f(x).$$

□

Then, the ridge regression estimate of $f \in S^2[0, 1]$ is

$$\hat{f}_\lambda \in \arg \min_{f \in S^2[0, 1]} \left\{ \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int_0^1 (f''(x))^2 dx \right\}$$

which exists and is unique and finite-dimensional. The solution is called a **smoothing spline**.

Note. $\lambda \rightarrow \infty$ gives a linear function, $\lambda \rightarrow 0$ gives an interpolating function (very wiggly).

3.2 Linear Basis Function Models

Consider the linear regression model:

$$Y_i = x_i^\top \beta + \epsilon_i, \quad \epsilon_i \stackrel{iid}{\sim} N(0, \sigma^2).$$

A linear basis function model models f as a linear combination of basis functions:

$$\mathbb{E}[Y | X = x] = \beta_0 + \sum_{j=1}^d h_j(x) \beta_j, \quad d \geq 1$$

and h_j are basis functions that are fixed and known.

Example. When $d = 1$ and choose $h_1(x) = x$, we have linear regression.

Example. When $d \geq 1$ and choose $h_j(x) = x^j$, we have polynomial regression.

Example. Let $I_j = (a_j, a_{j+1})$, $a_1 < a_2 < \dots < a_{d+1}$, and define

$$h_j(x) = 1(x \in I_j) = \begin{cases} 1, & x \in I_j \\ 0, & x \notin I_j \end{cases}$$

Then, the fitted function is constant on each interval I_j . We call this a “locally-constant regression”.

Example. Choosing $h_j(x) = K(x, x_j)$, $j = 1, \dots, n$, where K is a positive semi-definite kernel gives a function f from a RKHS.

3.2.1 Polynomial Regression

When polynomial degree is large, the fitted function can be very wiggly, which fits the training data well but behaves poorly on new data. This is called **overfitting**. Two explanations for this behavior:

- (1) The polynomial basis functions are **global**: every h_j is nonzero for all $x_i \neq 0$. This means that a small change in any x_i will affect the shape of the entire fitted function.
- (2) The polynomial basis functions are very **flexible** when the order is high: when the number of coefficients is close to n (i.e. the number of data points), the polynomial can interpolate the data, leading to extremely high variance in the shape of the fitted function.

There are two solutions that create a very useful tool for fitting nonlinear regression functions, when applied together.

3.2.2 Piecewise Polynomial Regression

The first solution is to remove the global property of the basis by using **piecewise polynomials**.

Knots: $\tau_0 \leq \dots \leq \tau_d$ are chosen boundary points that split the domain of x into d intervals:

$$[\tau_0, \tau_1), [\tau_1, \tau_2), \dots, [\tau_{d-1}, \tau_d)$$

Piecewise Polynomial: a function that is a polynomial on each interval $[\tau_j, \tau_{j+1})$.

Example. Piecewise polynomial with 5 knots has 4 intervals, and hence 4 polynomials.

Example. Order 1/degree 0 piecewise polynomial: $f(x) = \sum_{j=1}^d h_j(x)$ where

$$h_j(x) = \mathbb{1}(x \in [\tau_{j-1}, \tau_j)) \beta_j = \begin{cases} \beta_j, & x \in [\tau_{j-1}, \tau_j) \\ 0, & x \notin [\tau_{j-1}, \tau_j). \end{cases}$$

Example. Order 2/degree 1 piecewise polynomial: $f(x) = \sum_{j=1}^d h_j(x)$ where

$$h_j(x) = \mathbb{1}(x \in [\tau_{j-1}, \tau_j)) (\alpha_j + \beta_j x).$$

Note. Order = number of polynomial terms used to fit each piece.

This solves the global problem as changes to $x \in [\tau_1, \tau_2)$ won't affect the fitted function on $[\tau_2, \tau_3)$, and so on. But this model is too flexible so does not fit the data well. There are jumps in the fitted function at the knots. To fix this, we restrict the piecewise polynomial basis of order p so that it has continuous derivatives of order $0, \dots, p-2$ at the knots.

Definition 3.8 (Spline).

A curve that satisfies the continuity conditions above is called a **spline**.

We can choose a basis that satisfies the continuity conditions. This is achieved by the **truncated power function basis**. For a fixed knot sequence $\tau_1 \leq \dots \leq \tau_d$,

$$h_j(x) = x^{j-1}, \quad j = 1, \dots, p$$

and

$$h_{p+\ell}(x) = (x - \tau_\ell)_+^{p-1}, \quad \ell = 1, \dots, d.$$

So, the fitted function is

$$\begin{aligned} f(x) &= \sum_{j=1}^{p+d} \beta_j h_j(x) \\ &= \beta_1 + \beta_2 x + \dots + \beta_p x^{p-1} + \sum_{\ell=1}^d \beta_{p+\ell} (x - \tau_\ell)_+^{p-1}. \end{aligned}$$

Proposition 3.4. The truncated power function basis satisfies the following properties:

- (1) The number of parameters is equal to the order of the polynomial basis plus the number of knots. That is, $m = p + d$.
- (2) h_{j+p} is a p th order polynomial on (τ_{j-1}, τ_j) for $j = 1, \dots, d$.
- (3) $f(x) = \sum_{j=1}^{p+d} h_j(x) \beta_j$ has $p - 2$ continuous derivatives at $x = \tau_j$ for $j = 1, \dots, d$.

The number of parameters can also be computed by subtracting the number of continuity constraints from the number of parameters in the piecewise polynomial basis:

The unconstrained piecewise polynomial basis has order p (i.e. p parameters per interval) and d knots (i.e. $d + 1$ intervals where we also include $(-\infty, \tau_1)$ and $[\tau_d, \infty)$). Hence, the number of parameters is $(d + 1)p$ in this unconstrained basis.

The continuity constraints require the function itself and its first $p - 2$ derivatives to be continuous at each of the d knots. This gives $d(p - 1)$ constraints.

Then, the number of parameters in the constrained basis is

$$(d + 1)p - d(p - 1) = dp + p - dp + d = p + d.$$

The functions $h_1, \dots, h_{p+d}$ span the space of order- p piecewise polynomials with knots $\tau_1, \dots, \tau_d$ and $p - 2$ continuous derivatives at the knots.

Example. Let $p = 4$. Then, the spline has degree 3 and the fitted function is

$$f(x) = \beta_1 + \beta_2 x + \beta_3 x^2 + \beta_4 x^3 + \sum_{\ell=1}^d \beta_{4+\ell} (x - \tau_\ell)_+^3.$$

Let's look at the truncated power function basis at a known τ , which is $(x - \tau)_+^3$. Let

$$g(x) = (x - \tau)_+^3 = \begin{cases} 0, & x < \tau \\ (x - \tau)^3, & x \geq \tau. \end{cases}$$

Note that g is continuous at $x = \tau$ because

$$\lim_{x \rightarrow \tau^-} g(x) = \lim_{x \rightarrow \tau^+} g(x) = 0 = g(\tau).$$

Now, check the first derivative:

$$g'(x) = \begin{cases} 0, & x < \tau \\ 3(x - \tau)^2, & x \geq \tau \end{cases} \implies \lim_{x \rightarrow \tau^-} g'(x) = \lim_{x \rightarrow \tau^+} g'(x) = 0 = g'(\tau).$$

Similarly,

$$g''(x) = \begin{cases} 0, & x < \tau \\ 6(x - \tau), & x \geq \tau \end{cases} \implies \lim_{x \rightarrow \tau^-} g''(x) = \lim_{x \rightarrow \tau^+} g''(x) = 0 = g''(\tau).$$

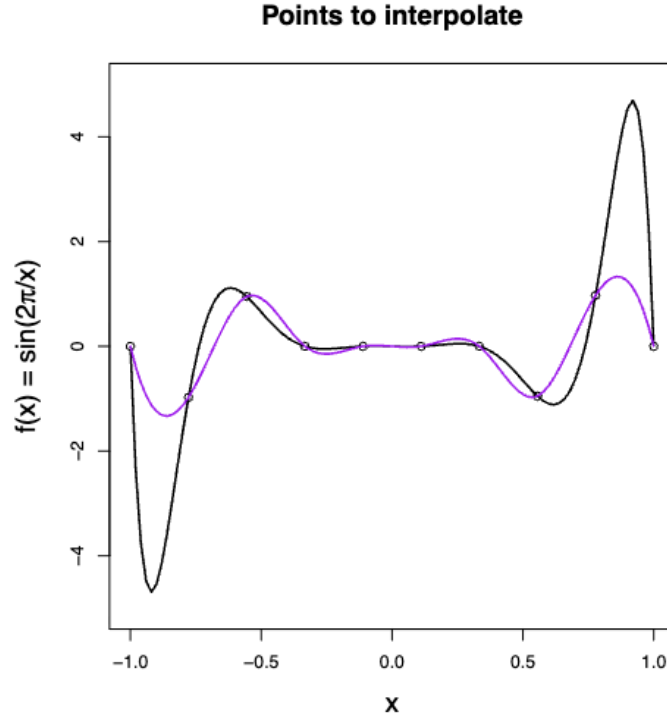
However, the third derivative is not continuous at $x = \tau$:

$$g'''(x) = \begin{cases} 0, & x < \tau \\ 6, & x \geq \tau \end{cases} \implies \lim_{x \rightarrow \tau^-} g'''(x) = 0 \neq 6 = \lim_{x \rightarrow \tau^+} g'''(x).$$

This matches with the fact that the spline has $p - 2 = 2$ continuous derivatives at the knots.

3.3 Splines

The following figure shows a Lagrange polynomial interpolant (black) and a cubic B-spline interpolant (purple). Both interpolate 10 points, but the polynomial is very wiggly between the points.



3.3.1 B-Splines

Let $a < b \in \mathbb{R}$ and $a = \underbrace{\tau_1 = \dots = \tau_p}_{p \text{ boundary knots}} \leq \underbrace{\tau_{1+p} \leq \dots \leq \tau_{d+p}}_{d \text{ interior knots}} \leq \underbrace{\tau_{d+p+1} = \dots = \tau_{d+2p}}_{p \text{ boundary knots}} = b$ be a knot sequence. Let $b_{j,p} : \mathbb{R} \rightarrow \mathbb{R}$ be the j th B-spline basis function of order p with knots $\tau_1, \dots, \tau_{d+2p}$. A **spline function** has the form

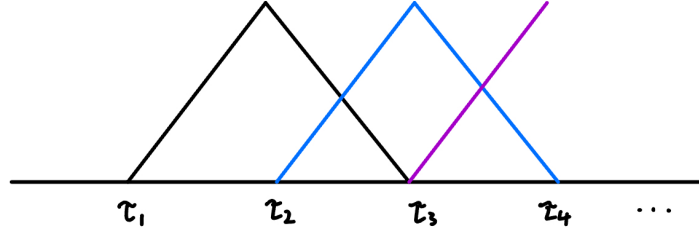
$$f(x) = \sum_{j=1}^{d+2p} b_{j,p}(x) \beta_j.$$

The recursive definition of the B-spline basis functions is as follows. Start with the order 1 indicator functions:

$$b_{j,1}(x) = \mathbb{1}(\tau_j \leq x < \tau_{j+1}) = \begin{cases} 1, & x \in [\tau_j, \tau_{j+1}) \\ 0, & x \notin [\tau_j, \tau_{j+1}) \end{cases}$$

We call $b_{i,1}$ the i th B-spline of order 1 (i.e. degree 0). Now, let

$$b_{j,2}(x) = \frac{x - \tau_j}{\tau_{j+1} - \tau_j} b_{j,1}(x) + \frac{\tau_{j+2} - x}{\tau_{j+2} - \tau_{j+1}} b_{j+1,1}(x).$$



Finally, for any $1 < p \in \mathbb{N}$, the i th B-spline of order p is

$$b_{j,p}(x) = \frac{x - \tau_j}{\tau_{j+p-1} - \tau_j} b_{j,p-1}(x) + \frac{\tau_{j+p} - x}{\tau_{j+p} - \tau_{j+1}} b_{j+1,p-1}(x).$$

Proposition 3.5 (Properties of B-Splines).

- (1) Local support: $b_{i,p}(x) = 0$ for $x \notin [\tau_i, \tau_{i+p}]$.
- (2) Positivity: $b_{i,p}(x) > 0 \forall x \in (\tau_i, \tau_{i+p})$.
- (3) Self-normalization: $\sum_{i=1}^{d+2p} b_{i,p}(x) = 1 \forall x \in [\tau_i, \tau_{i+p}]$.

Note. (3) means that at each x , the B-spline basis functions at that x add up to 1. This is useful for interpreting the fitted function as a weighted average of the coefficients β_j .

Definition 3.9 (Natural Cubic Spline).

A **natural cubic spline** is a spline of order 4 (i.e. degree 3) with knots $a = \tau_1, \dots, \tau_m = b$ that satisfies the additional condition that it is linear outside of the boundary knots. So, $f''(a) = f''(b) = 0$.

3.3.2 Smoothing Splines: Practical Construction

Splines are “calm” interpolants that interpolate the given set of points, but they seem to vary less than the polynomial interpolants between the points. It turns out that this is a fundamental property of splines. In fact, splines are characterized entirely by this property.

Consider the following measure of “roughness” of a twice-continuously differentiable function $f : \mathbb{R} \rightarrow \mathbb{R}$:

$$P(f) = \|f\|_2^2 = \int (f''(x))^2 dx.$$

Note. “Roughness” measures how wiggly/bendy a function is. A function with a lot of wiggles has a large second derivative, and hence a large roughness.

Now, consider the following optimization problem:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \left\{ \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int_0^1 (f''(u))^2 du \right\}$$

where $\mathcal{H} = S^2[0, 1]$. The second term is the roughness penalty, which penalizes large values of f'' .

Theorem 3.6 (Fundamental Theorem of Splines).

Suppose $n \geq 2$. Let $a \leq x_1 \leq \dots \leq x_n \leq b$ in $\mathbb{R}$. Let $g(x)$ be the unique natural cubic spline interpolant with $g(x_i) = y_i$ for $i = 1, \dots, n$. Then,

$$\hat{f} = g.$$

Note. This theorem says that among all twice-continuously differentiable functions that interpolate the data, the cubic spline interpolant is the one with the smallest roughness. Say $\tilde{g}$ is another twice-continuously differentiable function that interpolates the data. Then,

$$\int_a^b (\tilde{g}''(u))^2 du \geq \int_a^b (g''(u))^2 du.$$

Proof. Let $\tilde{g}$ be another twice-continuously differentiable function such that $\tilde{g}(x_i) = y_i$ for $i = 1, \dots, n$. We want to show that

$$\int_a^b (\tilde{g}''(u))^2 du \geq \int_a^b (g''(u))^2 du.$$

Let $h = \tilde{g} - g$. Expand the roughness of $\tilde{g}$:

$$\begin{aligned} \int_a^b (\tilde{g}''(u))^2 du &= \int_a^b (g''(u) + h''(u))^2 du \\ &= \int_a^b (g''(u))^2 du + 2 \int_a^b g''(u)h''(u) du + \int_a^b (h''(u))^2 du. \end{aligned}$$

Clearly, $\int_a^b (h''(u))^2 du \geq 0$. So, it suffices to show that $\int_a^b g''(u)h''(u) du = 0$. Recall that a cubic

spline is linear for $x \notin (a, b)$, so $g''(a) = g''(b) = 0$. Now, integrate by parts:

$$\begin{aligned} \int_a^b g''(u)h''(u) du &= g''(u)h'(u) \Big|_a^b - \int_a^b g'''(u)h'(u) du \\ &= - \int_a^b g'''(u)h'(u) du \quad \text{since } g''(a) = g''(b) = 0. \end{aligned}$$

Now, note that g''' is constant on each interval (x_i, x_{i+1}) for $i = 1, \dots, n-1$. So, we can write

$$\begin{aligned} \int_a^b g'''(u)h'(u) du &= \sum_{i=1}^{n-1} \int_{x_i}^{x_{i+1}} g'''(u)h'(u) du \\ &= \sum_{i=1}^{n-1} g'''(u) \int_{x_i}^{x_{i+1}} h'(u) du \\ &= \sum_{i=1}^{n-1} g'''(u)(h(x_{i+1}) - h(x_i)) \\ &= \sum_{i=1}^{n-1} g'''(u)(\tilde{g}(x_{i+1}) - g(x_{i+1}) - (\tilde{g}(x_i) - g(x_i))) \\ &= \sum_{i=1}^{n-1} g'''(u)(y_{i+1} - y_{i+1} - (y_i - y_i)) = 0. \end{aligned}$$

Thus,

$$\int_a^b (\tilde{g}''(u))^2 du = \int_a^b (g''(u))^2 du + \underbrace{0 + \int_a^b (h''(u))^2 du}_{\geq 0} \geq \int_a^b (g''(u))^2 du.$$

To see that the minimizer $\hat{f}$ must be a cubic spline, suppose $\hat{f}$ is not a cubic spline and it interpolates the data. Then, let g be a cubic spline such that $g(x_i) = \hat{f}(x_i)$ for $i = 1, \dots, n$. Then, by the above argument,

$$\int_a^b (g''(u))^2 du \leq \int_a^b (\hat{f}''(u))^2 du.$$

But, $\sum_{i=1}^n (y_i - g(x_i))^2 = \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$ since g and $\hat{f}$ interpolate the same data. Hence, the objective function value of g is less than or equal to that of $\hat{f}$, which contradicts the optimality of $\hat{f}$. Hence, $\hat{f}$ must be a cubic spline and $\hat{f} = g$. $\square$

Let's summarize and give the definition of a spline again.

Definition 3.10 (Spline (revisited)).

A **spline** of order $p \in \mathbb{N}$ with knots $\tau_1 \leq \dots \leq \tau_d$ is a piecewise polynomial of order p (i.e. degree $p - 1$). That is, a function that is

- (a) a polynomial of order p on each interval $(-\infty, \tau_1), (\tau_1, \tau_2), \dots, (\tau_{d-1}, \tau_d), (\tau_d, \infty)$;
- (b) $p - 2$ times continuously differentiable at each knot τ_j for $j = 1, \dots, d$.

Note. If we force the spline to be $p - 1$ times continuously differentiable at each knot, then it would become a global polynomial.

To see this, note that adjacent polynomial pieces would agree on the value and the first $p - 1$ derivatives at each knot. Hence, the two polynomial pieces would be the same polynomial, and the spline would be a single polynomial.

To obtain a spline, we use the truncated power function basis so the fitted function is

$$f(x) = \sum_{j=1}^p \beta_j x^{j-1} + \sum_{\ell=1}^d \beta_{p+\ell} (x - \tau_\ell)_+^{p-1}.$$

3.4 Regression Splines

Now, we can fit a nonlinear regression model by choosing some knots, modelling the regression function as a spline function with those knots, and then estimating the weights by least squares. This is called a **regression spline**.

We model:

$$\mathbb{E}[Y | X = x] = \sum_{j=1}^p \beta_j x^{j-1} + \sum_{\ell=1}^d \beta_{p+\ell} (x - \tau_\ell)_+^{p-1}.$$

For observation i , the i th row of the design matrix is

$$X_i = \begin{bmatrix} 1 & x_i & x_i^2 & \dots & x_i^{p-1} & (x_i - \tau_1)_+^{p-1} & \dots & (x_i - \tau_d)_+^{p-1} \end{bmatrix}.$$

Then, the model becomes

$$Y = X\beta + \epsilon, \quad \epsilon \sim N(0, \sigma^2 I)$$

where $\beta = (\beta_1, \dots, \beta_{p+d})^\top$. The least squares estimate is just

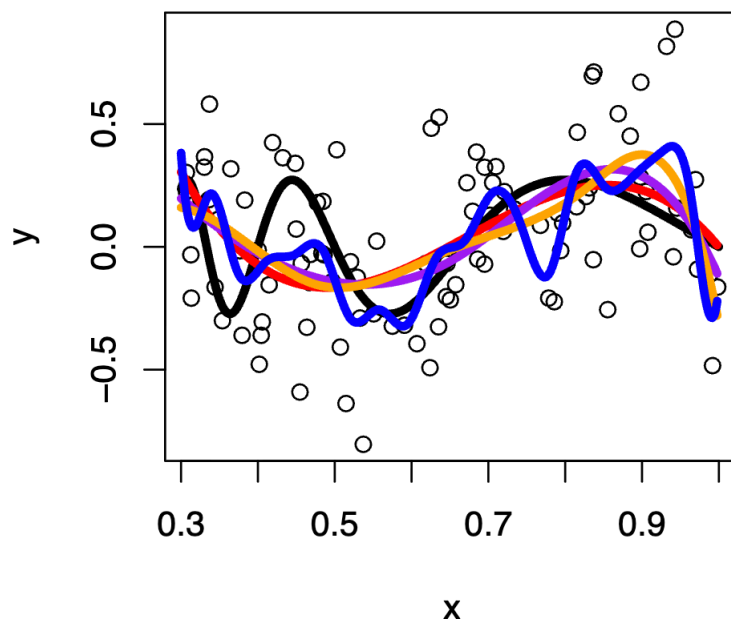
$$\hat{\beta} = (X^\top X)^{-1} X^\top Y.$$

Also,

$$\text{Var}(\hat{\beta}) = \sigma^2(X^\top X)^{-1} \quad \text{and} \quad \hat{\sigma}^2 = \frac{1}{n - (p + d)} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2.$$

The choice of knots matters. They are the points where the fitted function is allowed to change its shape. If we choose too few knots, the fitted function will be **biased** and not flexible enough to capture the shape of the true regression function. If we choose too many knots, the fitted function will be **high variance** and too wiggly/rough.

B-Spline regression



This figure shows a B-spline regression for a nonlinear regression function (black) with 2 (red), 3 (purple), 5 (orange), and 20 (blue) knots.

We need to penalize complexity/roughness of the fitted function!

3.4.1 Fitting a Spline by Ridge Regression

The model is (in matrix form):

$$Y = X\beta + \epsilon, \quad \epsilon \sim N(0, \sigma^2 I)$$

where X is the spline design matrix, which can either be the truncated polynomials (above) or the B-splines, i.e. $X_{ij} = b_{j,p}(x_i) = b_j(x_i)$ where $b_{j,p}$ is the j th B-spline basis function.

Regression splines fix f to be a cubic spline (i.e. $p = 4$) with some chosen large number of knots.

The model is

$$Y_i = f(x_i) + \epsilon_i.$$

The fitted function is

$$f(x) = \sum_{j=1}^p \beta_j x^{j-1} + \sum_{\ell=1}^d \beta_{p+\ell} (x - \tau_\ell)_+^{p-1}.$$

Or,

$$f(x) = \sum_{j=1}^{p+d} \beta_j b_j(x).$$

The same spline function f can be represented in either basis. Then, choose the spline weights to minimize the penalized least squares objective:

$$\hat{\beta} = \arg \min_{\beta} \{ \ell(\beta; y) + P(\beta; \lambda) \}$$

where

$$\ell(\beta; y) = \frac{1}{\sigma^2} \sum_{i=1}^n (y_i - f(x_i))^2 \quad \text{and} \quad P(\beta; \lambda) = \lambda \int (f''(u))^2 du \quad \text{for some } \lambda > 0.$$

We rewrite the penalty term as a quadratic form in β :

$$\begin{aligned} P(\beta; \lambda) &= \lambda \int (f''(u))^2 du = \lambda \int (B''(u)^\top \beta)^2 du \quad \text{where } B''(u) = (b_1''(u), \dots, b_{p+d}''(u))^\top \\ &= \lambda \int \beta^\top B''(u) B''(u)^\top \beta du \\ &= \lambda \beta^\top \underbrace{\int B''(u) B''(u)^\top du}_{\text{matrix } S} \beta \\ &= \lambda \beta^\top S \beta. \end{aligned}$$

where $S \in \mathbb{R}^{(p+d) \times (p+d)}$ and $\text{rank}(S) = \dim(S) - 2$. Note that $S_{k\ell} = \int b_k''(u) b_\ell''(u) du$.

Note. If f is linear, then $f'' = 0$ and hence $P(\beta; \lambda) = 0$.

Then, write the objective function in matrix form and solve for $\hat{\beta}$:

$$\ell(\beta; y) = \frac{1}{\sigma^2} (y - X\beta)^\top (y - X\beta) \quad \text{and} \quad P(\beta; \lambda) = \lambda \beta^\top S \beta.$$

We differentiate the objective function with respect to β and set it equal to 0:

$$\begin{aligned}
\frac{\partial}{\partial \beta} \{ \ell(\beta; y) + P(\beta; \lambda) \} &= \frac{\partial}{\partial \beta} \left\{ \frac{1}{\sigma^2} (y - X\beta)^\top (y - X\beta) + \lambda \beta^\top S \beta \right\} = 0 \\
\frac{\partial}{\partial \beta} \left\{ \frac{1}{\sigma^2} (y^\top y - y^\top X\beta - \beta^\top X^\top y + \beta^\top X^\top X\beta) + \lambda \beta^\top S \beta \right\} &= 0 \\
\frac{\partial}{\partial \beta} \left\{ \frac{1}{\sigma^2} (y^\top y - 2\beta^\top X^\top y + \beta^\top X^\top X\beta) + \lambda \beta^\top S \beta \right\} &= 0 \\
\Rightarrow \frac{1}{\sigma^2} (-2X^\top y + 2X^\top X\beta) + 2\lambda S\beta &= 0 \quad \text{since } S \text{ is symmetric} \\
\Rightarrow X^\top X\beta + \lambda \sigma^2 S\beta &= X^\top y \\
\Rightarrow \hat{\beta} &= (X^\top X + \lambda \sigma^2 S)^{-1} X^\top y.
\end{aligned}$$

This is similar to the ridge regression solution, except that the penalty term is not just λI but $\lambda \sigma^2 S$.

Proposition 3.7 (Sampling Distribution of $\hat{\beta}$).

Given the model $Y \sim N(X\beta, \sigma^2 I)$ where $X\beta = (f(x_1), \dots, f(x_n))^\top$ and f is a spline function,

- (1) $E[\hat{\beta}] = (X^\top X + \lambda \sigma^2 S)^{-1} X^\top X\beta$.
- (2) $\text{Var}(\hat{\beta}) = \sigma^2 (X^\top X + \lambda \sigma^2 S)^{-1} X^\top X (X^\top X + \lambda \sigma^2 S)^{-1}$.
- (3) $\hat{\beta} \sim N(E[\hat{\beta}], \text{Var}(\hat{\beta}))$.

Proof.

- (1) $E[\hat{\beta}] = E[(X^\top X + \lambda \sigma^2 S)^{-1} X^\top Y] = (X^\top X + \lambda \sigma^2 S)^{-1} X^\top E[Y] = (X^\top X + \lambda \sigma^2 S)^{-1} X^\top X\beta$.
- (2) $\text{Var}(\hat{\beta}) = \text{Var}((X^\top X + \lambda \sigma^2 S)^{-1} X^\top Y) = (X^\top X + \lambda \sigma^2 S)^{-1} X^\top \text{Var}(Y) X (X^\top X + \lambda \sigma^2 S)^{-1} = \sigma^2 (X^\top X + \lambda \sigma^2 S)^{-1} X^\top X (X^\top X + \lambda \sigma^2 S)^{-1}$.
- (3) $\hat{\beta}$ is a linear transformation of Y , which also follows N .

□

The fitted function is $\hat{f}(x) = \sum_{j=1}^{p+d} b_j(x) \hat{\beta}_j \equiv b(x)^\top \hat{\beta}$ where $b(x) = (b_1(x), \dots, b_{p+d}(x))^\top$. For any new points $x^* = (x_1^*, \dots, x_N^*)^\top$, let $\hat{f}(x^*) = (\hat{f}(x_1^*), \dots, \hat{f}(x_N^*))^\top$. Then,

$$\hat{f}(x^*) = B(x^*) \hat{\beta}, \quad \text{where } B(x^*) = \begin{bmatrix} b_1(x_1^*) & \cdots & b_{p+d}(x_1^*) \\ \vdots & \ddots & \vdots \\ b_1(x_N^*) & \cdots & b_{p+d}(x_N^*) \end{bmatrix} \in \mathbb{R}^{N \times (p+d)}.$$

Thus, the sampling distribution of $\hat{f}(x^*)$ is

$$\hat{f}(x^*) \sim N(B(x^*)\mathbb{E}[\hat{\beta}], B(x^*)\text{Var}(\hat{\beta})B(x^*)^\top).$$

The j th fitted value is

$$\hat{f}(x_j^*) = [\hat{f}(x^*)]_j \sim N(b(x_j^*)^\top \mathbb{E}[\hat{\beta}], b(x_j^*)^\top \text{Var}(\hat{\beta}) b(x_j^*)).$$

An approximate 95% confidence interval for $\hat{f}(x_j^*)$ is

$$\hat{f}(x_j^*) \pm z_{0.975} \sqrt{b(x_j^*)^\top \text{Var}(\hat{\beta}) b(x_j^*)}, \quad \text{where } z_{0.975} \text{ is s.t. } \mathbb{P}(Z \leq z_{0.975}) = 0.975 \text{ for } Z \sim N(0, 1).$$

Note that $\hat{Y} = X\hat{\beta} = X(X^\top X + \lambda^* \sigma^2 S)^{-1} X^\top Y \equiv H(\lambda^*)Y$. To estimate the residual variance σ^2 , we use

$$\hat{\sigma}^2 = \frac{1}{n - \text{tr}(H(\lambda^*))} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2,$$

where $\text{tr}(H(\lambda^*))$ is the number of fitted parameters. This is the **effective degrees of freedom**. Usually, $\text{tr}(H(\lambda^*)) \ll d + p$ because the penalty shrinks the wiggly directions of the fitted function. So,

- λ^* small $\implies \text{tr}(H(\lambda^*))$ large.
- λ^* large $\implies \text{tr}(H(\lambda^*))$ small.

Example. Consider cubic smoothing splines, as $\lambda^* \rightarrow \infty$, the fitted function $\hat{f}$ approaches a straight line, so the effective degrees of freedom $\text{tr}(H(\lambda^*)) \rightarrow 2$.

3.4.2 Choosing λ

One way to choose λ is to explicitly balance fit and complexity. We choose $\hat{\lambda}$ to minimize:

$$\underbrace{\sum_{i=1}^n (y_i - \hat{f}(x_i))^2}_{\text{fit}} + \underbrace{\text{tr}(H(\lambda))}_{\text{complexity}}.$$

This tries to minimize the same criterion that CV minimizes, i.e. the average generalization error.

Thus, we can also choose λ by minimizing the CV score:

$$\begin{aligned} \text{CV} : \widehat{\text{CV}}(\lambda) &= \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - (H(\lambda))_{ii}} \right)^2 \\ \text{GCV} : \widehat{\text{GCV}}(\lambda) &= \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - \text{tr}(H(\lambda))/n} \right)^2. \end{aligned}$$

3.4.3 Penalty Nullspace

In ridge regression, the coefficients were shrunk towards zero. The penalized splines problem reduces to ridge regression if the penalty matrix S is taken to be the identity matrix:

$$P(\beta; \lambda) = \lambda \beta^\top S \beta \implies \lambda \beta^\top \beta \text{ if } S = I.$$

The nullspace of the penalty matrix determines which class of functions is unpenalized, and hence which type of function we are smoothing towards. For ridge regression, the identity is full rank so the nullspace is $\{0\}$, and we smooth towards the zero function. For penalized splines, linear functions are unpenalized,

$$P(f; \lambda) = \lambda \int (f''(u))^2 du = P(f^*; \lambda) \quad \text{where } f^*(x) = \alpha + \beta x, \quad \alpha, \beta \in \mathbb{R}.$$

The penalty matrix has rank $p + d - 2$ and the nullspace is spanned by $\{1, x\}$.

Note. The nullspace of S is $\{v : Sv = \vec{0}\}$.

Therefore, we interpret the smoothing parameter λ as controlling how nonlinear deviation from a linear function we allow. If λ is large, we penalize to pull f closer to a linear function. If λ is small, we allow more nonlinear deviation from a linear function.

Geometric Interpretation of the Penalty

From the eigen-decomposition of S , we can write

$$\begin{aligned} S &= U \Lambda U^\top \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_{p+d-2}, 0, 0) \\ &= U_R \Lambda_+ U_R^\top \quad \Lambda_+ = \text{diag}(\lambda_1, \dots, \lambda_{p+d-2}) \end{aligned}$$

where we partition the eigenvectors into $U = [U_R \ U_F]$ where $U_R \in \mathbb{R}^{(p+d) \times (p+d-2)}$ and $U_F \in$

$\mathbb{R}^{(p+d) \times 2}$. Then,

$$\beta^\top S \beta = (U_R^\top \beta)^\top \Lambda_+ (U_R^\top \beta).$$

Now, split the spline design matrix:

$$\begin{aligned} X\beta &= XU U^\top \beta = X(U_R U_R^\top + U_F U_F^\top) \beta \\ &= XU_R U_R^\top \beta + XU_F U_F^\top \beta \\ &= XU_R \beta_R + XU_F \beta_F \quad \text{where } \beta_R = U_R^\top \beta, \beta_F = U_F^\top \beta \\ &= X_R \beta_R + X_F \beta_F \quad \text{where } X_R = XU_R, X_F = XU_F. \end{aligned}$$

The model becomes

$$Y = \underbrace{X_R \beta_R}_{\text{penalized}} + \underbrace{X_F \beta_F}_{\text{unpenalized}} + \epsilon.$$

Note. X_R can have collinear columns and X_F must have linearly independent columns.

This shows that the penalty does not shrink the entire spline towards zero. It shrinks only the non-linear part towards zero, thereby pulling the fitted function towards a linear function.

4 High-Dimensional Regression

Consider the linear regression model:

$$Y = X\beta + \epsilon \quad \text{or equivalently} \quad Y_i = \beta_0 + \beta_1 X_{i1} + \cdots + \beta_p X_{ip} + \epsilon_i.$$

There are n data points and p covariates (hence p regression coefficients). The term **high-dimensional** has two meanings:

- (1) $p \gg n$ or otherwise p ($p < n$) is impractically large for some actual data set.
- (2) $p \equiv p(n)$ depends on (increases with) n .

This chapter deals with the first case. The LASSO will give a method that results in a model depending only on a small number of covariates (variable selection) with a computational burden that depends on the number of covariates included in the model (not the total number you input). Regression trees give a model that can capture very high-dimensional interactions while being as simple and interpretable as a linear regression.

4.1 Least Absolute Shrinkage and Selection Operator (LASSO)

Ridge regression can shrink the influence of an input arbitrarily close to zero, but it cannot shrink it to exactly zero, and hence cannot actually remove an input from the model. We may wish to remove inputs if we want a sparser model, for reasons of computation, interpretation, and so on.

One solution is to remove inputs with influence below a given threshold. However, in the case of a very large number of inputs, most of which have nearly zero influence, this is very computationally wasteful. Also, it is not clear how to choose the threshold. Furthermore, dropping one input will change the singular values of another, so this process would have to be iterated until the singular values were all above the threshold.

Definition 4.1 (LASSO).

The **LASSO** is a method that performs both shrinkage and variable selection.

Note. The LASSO is extremely efficient in high-dimensional and sparse prediction problems because algorithms exist for computing the LASSO solution that scale with the number of non-zero parameters, rather than the total number of parameters.

4.1.1 L_1 -Penalties and Variable Selection

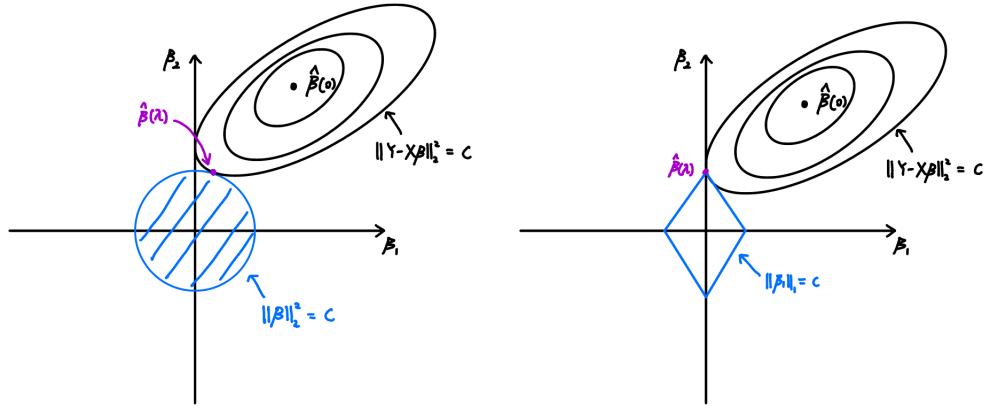
Consider the following optimization problems:

$$\text{Ridge : } \hat{\beta}(\lambda) = \arg \min_{\beta \in \mathbb{R}^p} \left\{ \|Y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \right\},$$

$$\text{LASSO : } \hat{\beta}(\lambda) = \arg \min_{\beta \in \mathbb{R}^p} \left\{ \|Y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\}$$

where $\|\beta\|_1 = \sum_{j=1}^p |\beta_j|$ is the L_1 -norm of β .

Changing from the 2-norm to the 1-norm can then set coefficients to exactly zero. Here is a geometric explanation:



The solution $\hat{\beta}(\lambda)$ is the point where the smallest black ellipse (the contour of the least squares loss) touches the constraint region (the blue region).

Note that the ridge uses the smooth circular region, while the LASSO uses the diamond-shaped region. For the LASSO, the smallest black ellipse touching the diamond is likely to touch at a corner, which corresponds to a coefficient being exactly zero.

4.1.2 Coordinate Descent and Sparsity

The LASSO problem no longer has an exact solution because the penalty term is not differentiable at zero.

For a general optimization problem, we want to minimize a convex function:

$$f : \mathbb{R}^p \rightarrow \mathbb{R}, \quad \hat{x} = \arg \min_{x \in \mathbb{R}^p} f(x).$$

Coordinate descent is a method that iteratively optimizes one coordinate at a time. Starting from $x = (x_1, \dots, x_p)$:

$$\begin{aligned}\hat{x}_1 &= \arg \min_{u \in \mathbb{R}} f(u, x_2, \dots, x_p), \\ \hat{x}_2 &= \arg \min_{u \in \mathbb{R}} f(\hat{x}_1, u, x_3, \dots, x_p) \quad \text{using updated } \hat{x}_1, \\ &\vdots \\ \hat{x}_p &= \arg \min_{u \in \mathbb{R}} f(\hat{x}_1, \dots, \hat{x}_{p-1}, u).\end{aligned}$$

This process is repeated until convergence, i.e. $x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_p \rightarrow x_1 \rightarrow \dots$ until the updates are sufficiently small.

This method is inefficient for general convex optimization problems, however, it is currently the best algorithm for computing the LASSO solutions. Here are some reasons:

- (1) The coordinate updates are in closed form and depend on quantities that can be pre-computed, enabling extremely fast iterations.
- (2) It can be calculated before each iteration which coefficients will be unchanged, and before the whole algorithm which coefficients will be zero.
- (3) The solutions for nearby values of λ are similar. After computing the solution for a given λ , we can use it as a starting point for the next value of λ .

Consider the objective:

$$\begin{aligned}\mathcal{L}_\lambda(\beta) &= \|Y - X\beta\|_2^2 + \lambda \|\beta\|_1 \\ &= \sum_{i=1}^n (y_i - x_i^\top \beta)^2 + \lambda \sum_{j=1}^p |\beta_j|.\end{aligned}$$

The partial derivative with respect to β_j is

$$\frac{\partial \mathcal{L}_\lambda}{\partial \beta_j} = -2 \sum_{i=1}^n x_{ij} (y_i - x_i^\top \beta) + \begin{cases} \lambda & \text{if } \beta_j > 0, \\ -\lambda & \text{if } \beta_j < 0, \\ \text{undefined} & \text{if } \beta_j = 0. \end{cases}$$

Assume that

$$\sum_{i=1}^n y_i = \sum_{i=1}^n x_{ij} = 0 \quad \forall j, \quad \sum_{i=1}^n x_{ij}^2 = 1 \quad \forall j.$$

We can show that

$$\hat{\beta}_j = \sum_{i=1}^n x_{ij} \left(y_i - \sum_{\ell \neq j} x_{i\ell} \beta_\ell \right) + \begin{cases} -\lambda & \text{if } \hat{\beta}_j > 0, \\ \lambda & \text{if } \hat{\beta}_j < 0. \end{cases}$$

Let the **soft-thresholding operator** be defined as

$$S(\alpha, \gamma) = \begin{cases} \alpha - \gamma & \text{if } \alpha > 0 \text{ and } \gamma < |\alpha|, \\ \alpha + \gamma & \text{if } \alpha < 0 \text{ and } \gamma < |\alpha|, \\ 0 & \text{if } \gamma \geq |\alpha|. \end{cases}$$

Then, the coordinate update for $\hat{\beta}_j$ is

$$\hat{\beta}_j = S \left(\sum_{i=1}^n x_{ij} \left(y_i - \sum_{\ell \neq j} x_{i\ell} \beta_\ell \right), \lambda \right).$$

Define fitted value and residuals as

$$\hat{y}_i = x_i^\top \hat{\beta}, \quad r_i = y_i - \hat{y}_i.$$

Then,

$$\begin{aligned} \sum_{i=1}^n x_{ij} \left(y_i - \sum_{\ell \neq j} x_{i\ell} \beta_\ell \right) &= \sum_{i=1}^n x_{ij} r_i + \hat{\beta}_j \\ &= x_j^\top y - \sum_{k: |\hat{\beta}_k| > 0} x_j^\top x_k \hat{\beta}_k + \hat{\beta}_j. \end{aligned}$$

Thus, the update is

$$\hat{\beta}_j = S \left(x_j^\top y - \sum_{k: |\hat{\beta}_k| > 0} x_j^\top x_k \hat{\beta}_k + \hat{\beta}_j, \lambda \right),$$

where $\{k : |\hat{\beta}_k| > 0\}$ is the **active set**. Note that the quantities $x_j^\top y$ and $x_j^\top x_k$ can be pre-computed. It can also be shown that starting at $\beta = 0$, $|\hat{\beta}_j| > 0 \iff |x_j^\top y| > \lambda$. Hence, it is known whether a $\hat{\beta}_j$ will change at each iteration based on the quantities computed before the iteration, speeding up the algorithm.

Computing a sequence of solutions as a function of λ (i.e. **solution path**) can be done efficiently. We want $\hat{\beta}(\lambda_1), \dots, \hat{\beta}(\lambda_d)$ for pre-chosen values $\lambda_1 > \dots > \lambda_d$. The software starts at

$$\lambda_1 = \lambda_{\max} = \max_{j=1, \dots, p} |x_j^\top y|.$$

Recall that starting at $\beta = 0$, the coordinate update is $\hat{\beta}_j = S(x_j^\top y, \lambda)$. Then, the soft-thresholding operator gives zero whenever $\lambda \geq |x_j^\top y|$. So, the condition $\lambda_{\max} = \max_{j=1, \dots, p} |x_j^\top y|$ ensures that $\hat{\beta}(\lambda) = 0$ for all $\lambda \geq \lambda_{\max}$.

The computational trick is to use the coefficients from the previous iteration as a starting value for the next iteration. This is called **warm starts**. It works well because nearby values of λ have similar solutions, i.e. $\lambda_{k+1} \approx \lambda_k \implies \hat{\beta}(\lambda_{k+1}) \approx \hat{\beta}(\lambda_k)$.

It is also known that $\hat{\beta}$ is a piecewise linear function of λ . Let

$$\hat{\beta}(\lambda) = \arg \min_{\beta \in \mathbb{R}^p} \left\{ \|Y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\}.$$

As λ decreases, more $\hat{\beta}_j$ become non-zero. Suppose that we have $\lambda_0 \geq \lambda_1$ such that $\hat{\beta}_k(\lambda_1) \neq 0 \iff \hat{\beta}_k(\lambda_0) \neq 0$. Then, $\exists \gamma \in \mathbb{R}^p$ such that if $\lambda \in [\lambda_1, \lambda_0]$, then

$$\hat{\beta}(\lambda) = \hat{\beta}(\lambda_0) + (\lambda - \lambda_0)\gamma.$$

Set $\lambda = \lambda_1$ gives $\hat{\beta}(\lambda_1) = \hat{\beta}(\lambda_0) + (\lambda_1 - \lambda_0)\gamma$.

The values of $\hat{\beta}$ will be close if their values of λ are close. That is,

- (1) $\|\hat{\beta}(\lambda_1) - \hat{\beta}(\lambda_0)\|_2 = |\lambda_1 - \lambda_0| \|\gamma\|_2$.
- (2) If λ_1, λ_0 are close enough, then $\hat{\beta}(\lambda_1), \hat{\beta}(\lambda_0)$ have the same active set (the same non-zero coefficients).

Note. In practice, the `glmnet` implementation is extremely fast for very high-dimensional, sparse regression problems.

4.2 Regression Trees

Regression trees are useful for modelling interactions, especially when there are many covariates. Consider the following model with two covariates:

$$\mathbb{E}[Y | X = x] = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2.$$

To see what the interaction does, compare the prediction at $x_1 + 1$ with the prediction at x_1 :

$$\mathbb{E}[Y | X_1 = x_1 + 1, X_2 = x_2] - \mathbb{E}[Y | X_1 = x_1, X_2 = x_2] = \beta_1 + \beta_3 x_2.$$

The effect of increasing X_1 by one unit also depends on x_2 now.

Remark. With p covariates, there are

$$\binom{p}{0} + \binom{p}{1} + \dots + \binom{p}{p} = 2^p$$

possible terms in the model. However, high-dimensional interactions are impossible to interpret in practice.

Trees create interpretable high-dimensional interaction-based models.

Definition 4.2 (Tree).

A **tree** is a method which partitions the sample space and then assigns predicted values to observations with covariates falling into each partition.

Suppose a tree has m **leaves** (terminal nodes): $R_1, \dots, R_m \subset \mathbb{R}^p$. The tree model is

$$\mathbb{E}[Y | X = x] = \sum_{j=1}^m \beta_j \mathbb{1}(x \in R_j)$$

with $R_i \cap R_j = \emptyset$, $\bigcup_{j=1}^m R_j = \mathbb{R}^p$, and $x \in \mathbb{R}^p$.

Note that we need to estimate R_j 's, so it is not a linear basis function model. If the partitions were known, then this would be a linear basis function model with

$$\hat{\beta} = \arg \min_{\beta \in \mathbb{R}^m} \sum_{i=1}^n \left(y_i - \sum_{j=1}^m \beta_j \mathbb{1}(x_i \in R_j) \right)^2 \implies \hat{\beta}_j = \frac{1}{|R_j|} \sum_{i: x_i \in R_j} y_i = \bar{y}_{R_j}.$$

The partitions are unknown. We use a greedy approach based on recursive binary partitions. Define the half-planes:

$$R_1(j, s) = \{x : x_j \leq s\}, \quad R_2(j, s) = \{x : x_j > s\}$$

where $s \in \mathbb{R}$ and $j = 1, \dots, p$. Consider the following prediction criterion:

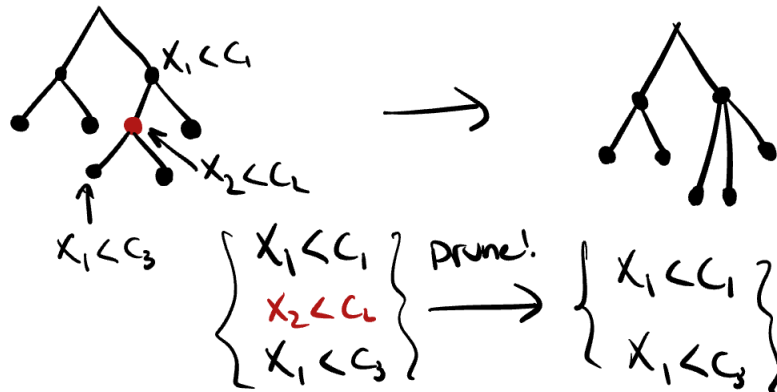
$$\min_{\substack{j=1, \dots, p \\ s \in \mathbb{R}}} \left[\min_{c_1} \sum_{x_i \in R_1(j, s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j, s)} (y_i - c_2)^2 \right].$$

For the inner minimization, we have

$$\hat{c}_k(j, s) = \text{average of } \{y_i : x_i \in R_k(j, s)\}, \quad k = 1, 2.$$

For the outer minimization, we can minimize over s numerically and over $j = 1, \dots, p$ directly by brute force. However, there is no guarantee that (j, s) is unique and that this procedure globally minimizes the training error (due to the greedy nature).

Growing a large tree will overfit the data. We can prune the tree to reduce the number of leaves. Let T_0 be the large tree obtained by applying recursive binary partitioning. A pruned tree $T \subseteq T_0$ is obtained by combining non-terminal nodes of T_0 .



Suppose T has m terminal nodes $R_1, \dots, R_m$. For region R_j , define

$$N_j = \#\{x_i : x_i \in R_j\} = \text{number of training observations in } R_j.$$

The prediction in R_j is

$$\hat{c}_j = \frac{1}{N_j} \sum_{x_i \in R_j} y_i.$$

Also, define

$$Q_j(T) = \frac{1}{N_j} \sum_{x_i \in R_j} (y_i - \hat{c}_j)^2 = \text{mean squared error in } R_j.$$

Then, the cost-complexity criterion is (total mean squared error + penalty for number of leaves):

$$C(T; \alpha) = \sum_{j=1}^m N_j Q_j(T) + \alpha m$$

Larger α gives more penalty for the number of leaves, resulting in a smaller tree (more pruning and fewer leaves).

4.3 More Topics

Previously, we have the following problem for the LASSO:

$$\arg \min_{\beta \in \mathbb{R}^p} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2 + \lambda \|\beta\|_1.$$

We want to get to the more general problem that `glmnet` solves:

- (1) Generalize the loss function to negative log-likelihood (allows for non-Gaussian error), i.e. $\sum_i \ell(y_i, \hat{f}(x_i))$.
- (2) Allow for observation weights, i.e. $\sum_i w_i (y_i - \hat{f}(x_i))$.
- (3) Allow for the whole range from least absolute to least squares shrinkage, i.e. $(a - \alpha) \|\beta\|_2^2 / 2 + \alpha \|\beta\|_1$.

More from the programming notes

Definition 4.3 (Random Forest).

A **random forest** is a method that extends regression or classification trees by

- taking a sub-sample of the observations and a sample of the variables, from the training set.

- fitting a regression tree to this sub-sample, which only has access to some observations and some variables.
- repeating this process many times to get many different trees.

Each of these trees is trained on different data and has access to different variables. So, each one should be independent enough. In the case of regression trees, we can combine the predictions of all the trees by an average or a weighted average. In the case of classification trees, each tree's classification of a response is a vote, and the final classification is the one with the most votes.

END OF STAT 444!